



# Deep Learning

Lecture 6.1: Attention Mechanisms and Transformers

# Today's Learning Objectives

**Be able to answer:**

- What is an attention mechanism?
- What is a transformer architecture?
  - How does scaled-dot-product attention work?
  - What are positional encodings?
- What are the shortcomings of transformers?
  - What directions are being pursued for efficient transformer models?





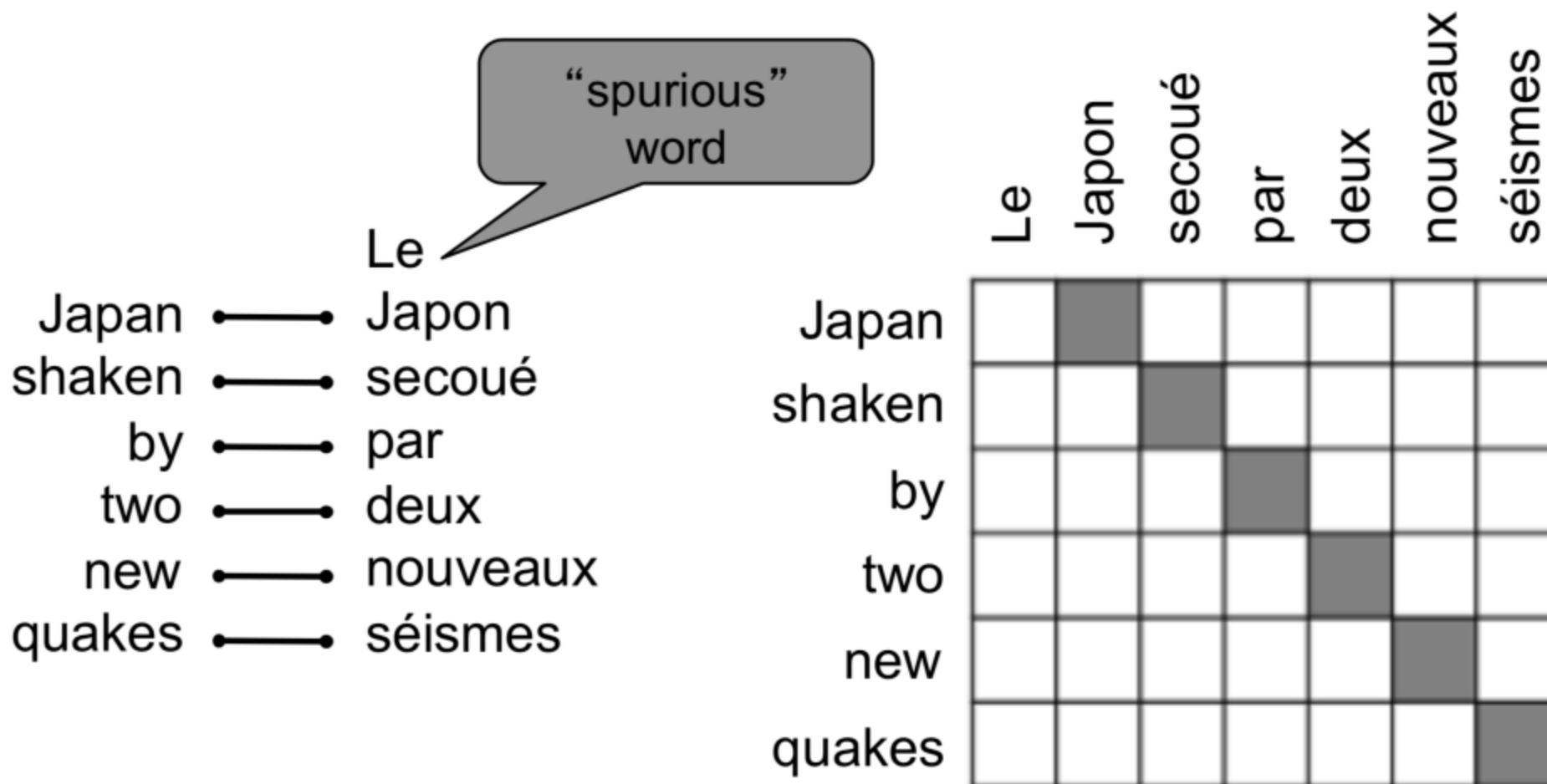
Japan shaken by two new quakes

Le Japon secoué par deux nouveaux séismes.



# Consider Machine Translation

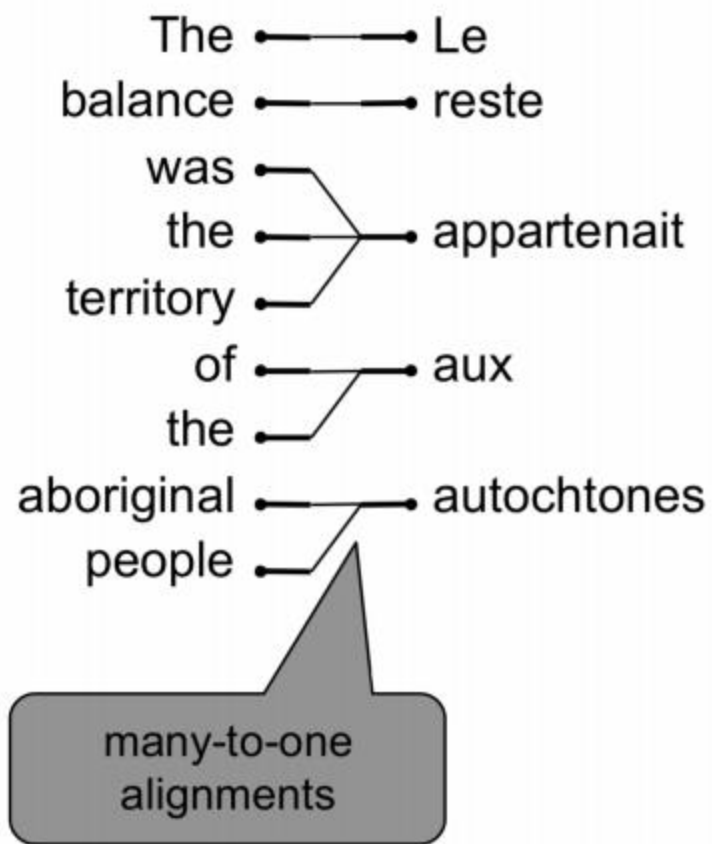
Some words have no alignment in the source language





# Consider Machine Translation

Some alignments are many-to-one

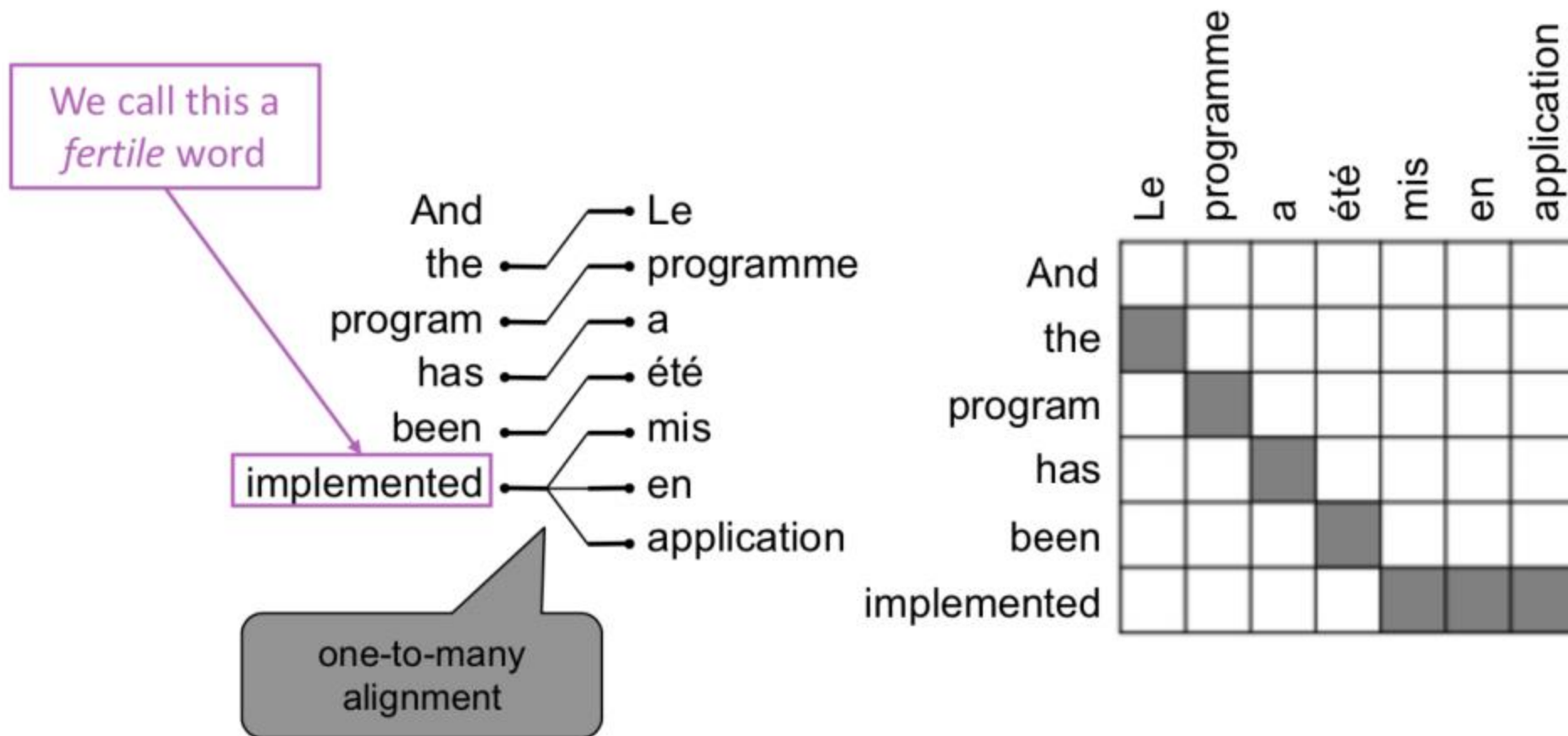


|            | Le | reste | appartenait | aux | autochtones |
|------------|----|-------|-------------|-----|-------------|
| The        |    |       |             |     |             |
| balance    |    |       |             |     |             |
| was        |    |       |             |     |             |
| the        |    |       |             |     |             |
| territory  |    |       |             |     |             |
| of         |    |       |             |     |             |
| the        |    |       |             |     |             |
| aboriginal |    |       |             |     |             |
| people     |    |       |             |     |             |



# Consider Machine Translation

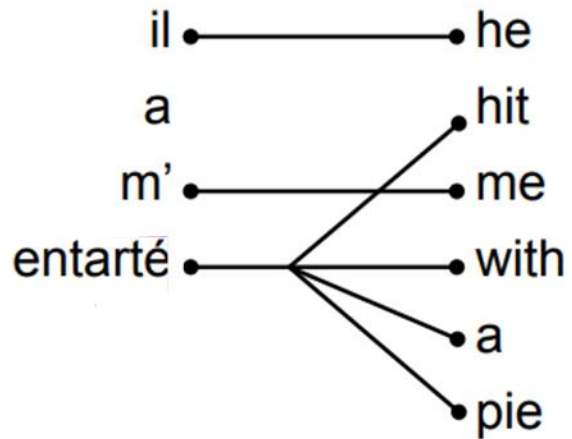
Others are one-to-many





# Consider Machine Translation

Some words are very fertile - especially words without direct translations



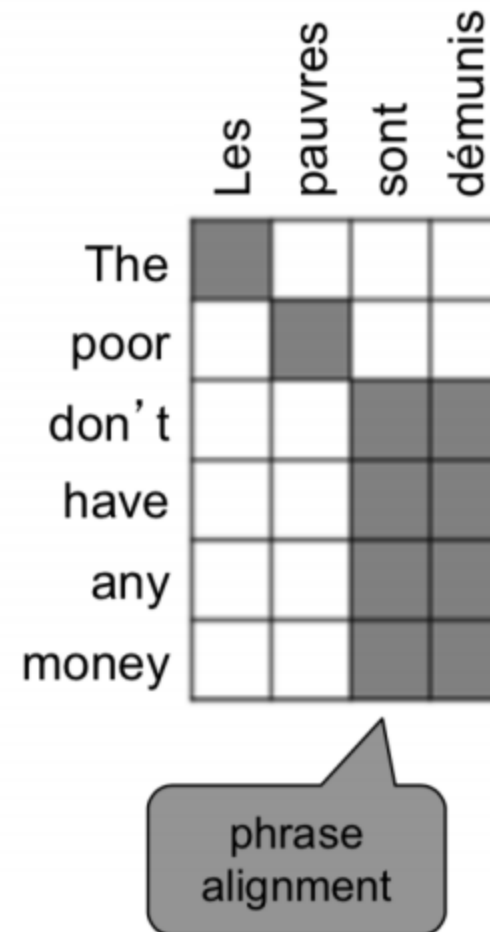
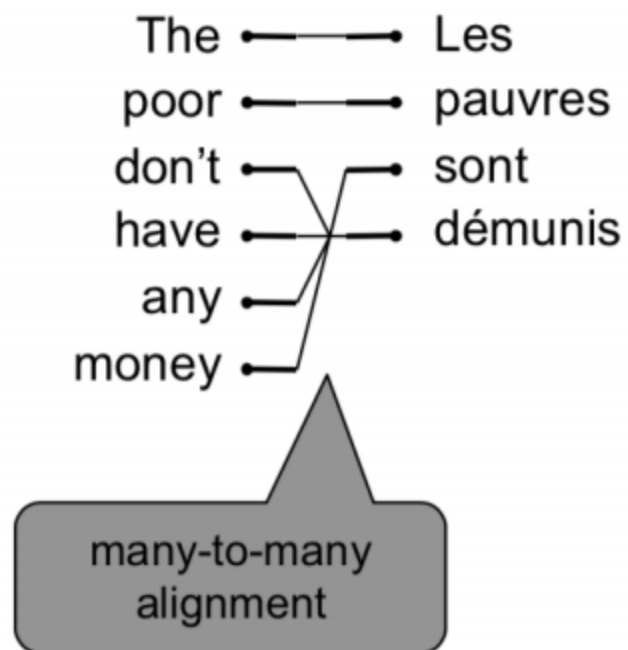
|         | he | hit | me | with | a | pie |
|---------|----|-----|----|------|---|-----|
| il      |    |     |    |      |   |     |
| a       |    |     |    |      |   |     |
| m'      |    |     |    |      |   |     |
| entarté |    |     |    |      |   |     |



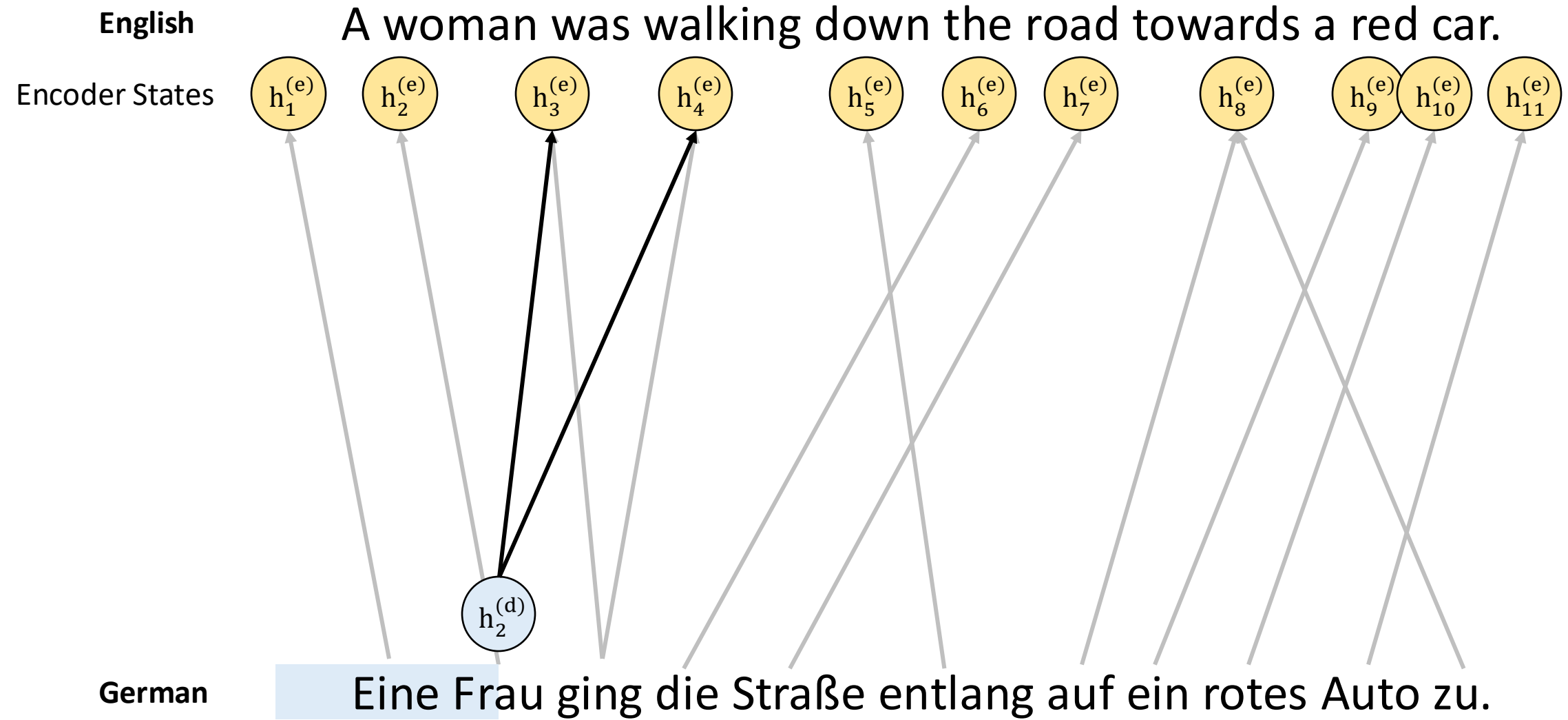


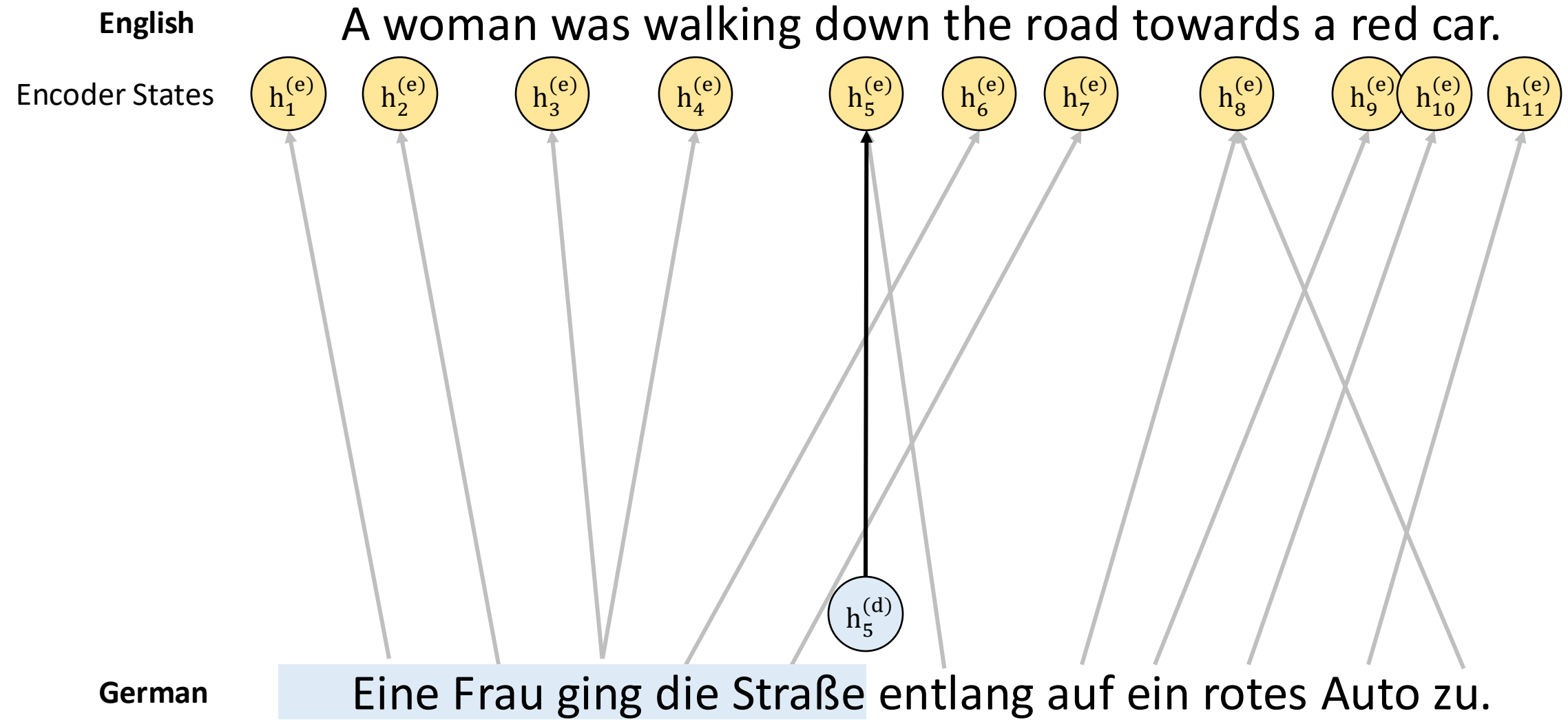
# Consider Machine Translation

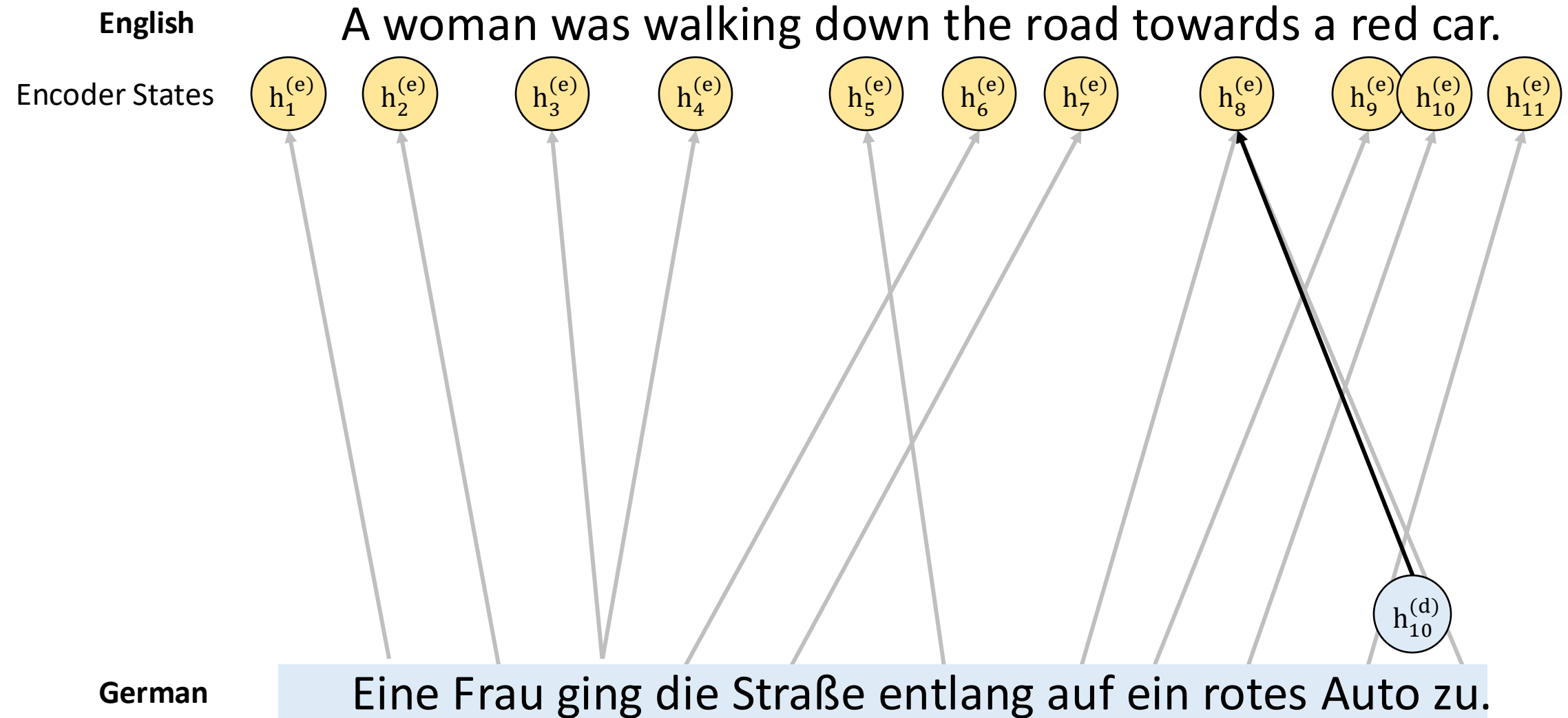
Phrase translations can be many-to-many













# How can we codify this notion of attention?

**Set of Candidates**

$v_1$   $v_2$   $v_3$   $v_4$   $v_5$  ...  $v_k$   $v_i \in \mathbb{R}^d$

**Query**

$q$   $q \in \mathbb{R}^c$

**Our Goal:** Some mechanism to select between  $k$  candidates given the query  $q$



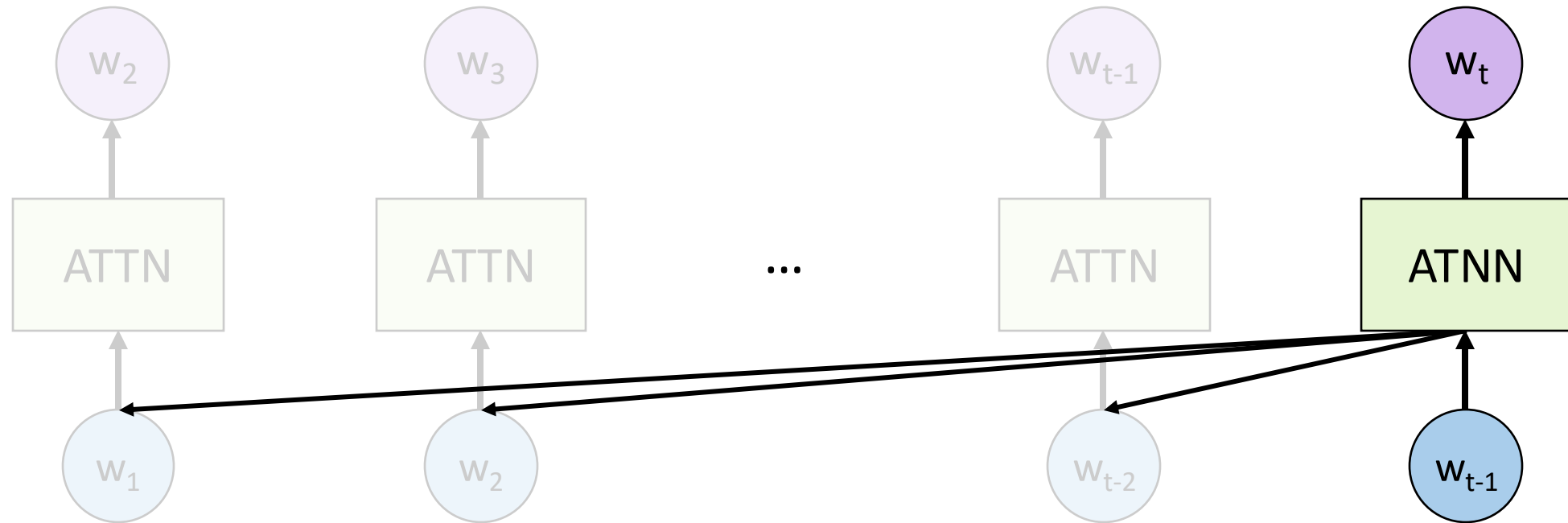
**Soft-attention** - This selection is a convex combination of candidates

**Hard-attention** - A true selection of only one candidate.



## Where we are going:

A model where everything attends to everything else.



# Today's Learning Objectives

Be able to answer:

- ~~What is an attention mechanism?~~
- What is a transformer architecture?
  - How does scaled-dot-product attention work?
  - What are positional encodings?
- What are the shortcomings of transformers?
  - What directions are being pursued for efficient transformer models?





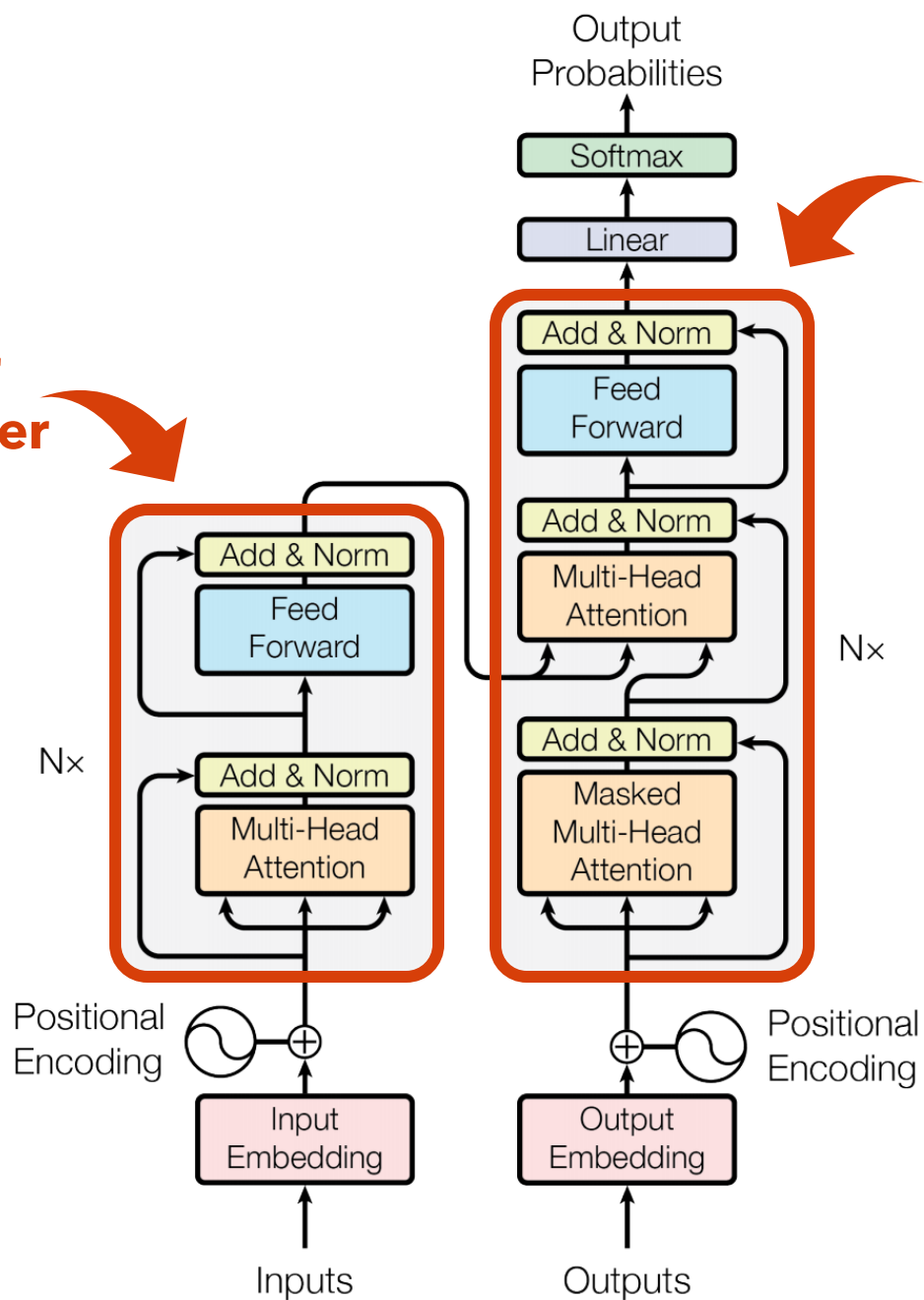
# Enter the Transformer



AI535

Image Credit: Vaswani et al, 2017.

**Encoder  
Transformer  
Block**



**Decoder  
Transformer  
Block**



## Attention Is All You Need, NeurIPS 2017.

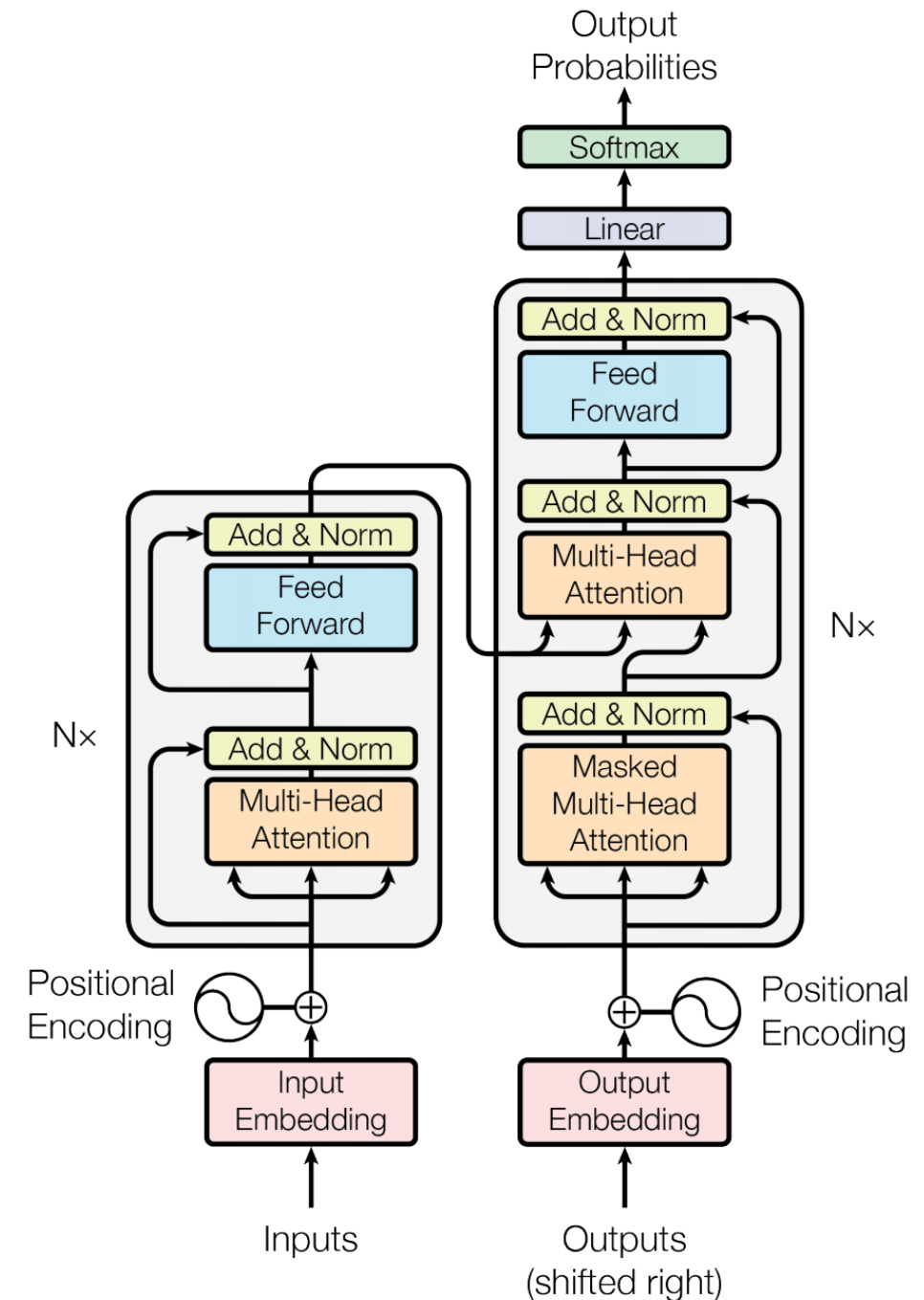
- Proposed an attention-based encoder-decoder architecture based on a novel *Transformer* self-attention block
- It works well on NLP benchmarks and is efficient to train
- Huge adoption in the community
- Increasing adoption in the vision community

[PDF] **Attention is all you need**

[A Vaswani](#) - Advances in Neural Information Processing Systems, 2017 - user.phil.hhu.de

**Attention is all you need** Attention is all you need ...

☆ Save 📄 Cite **Cited by 152179** Related articles »

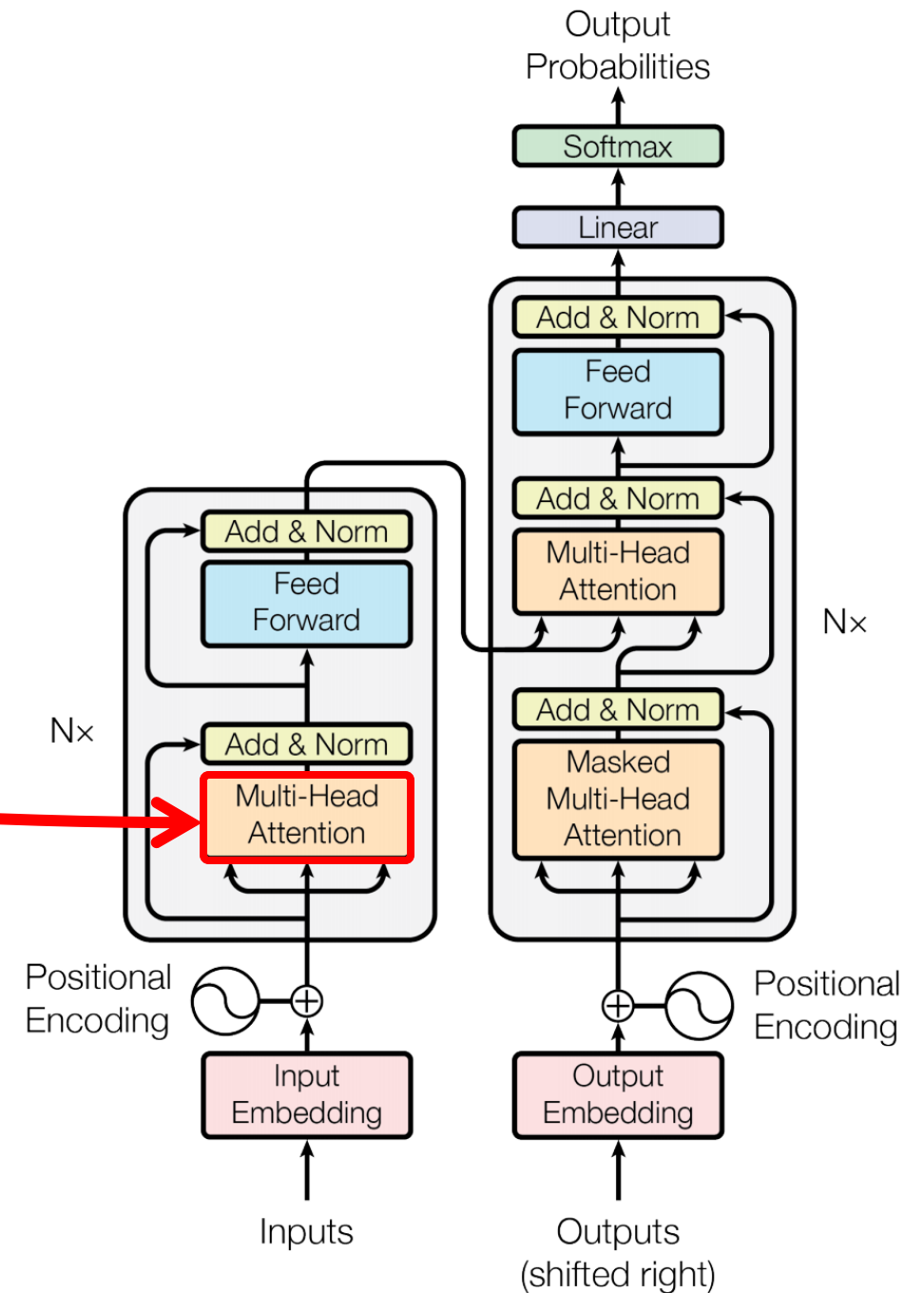


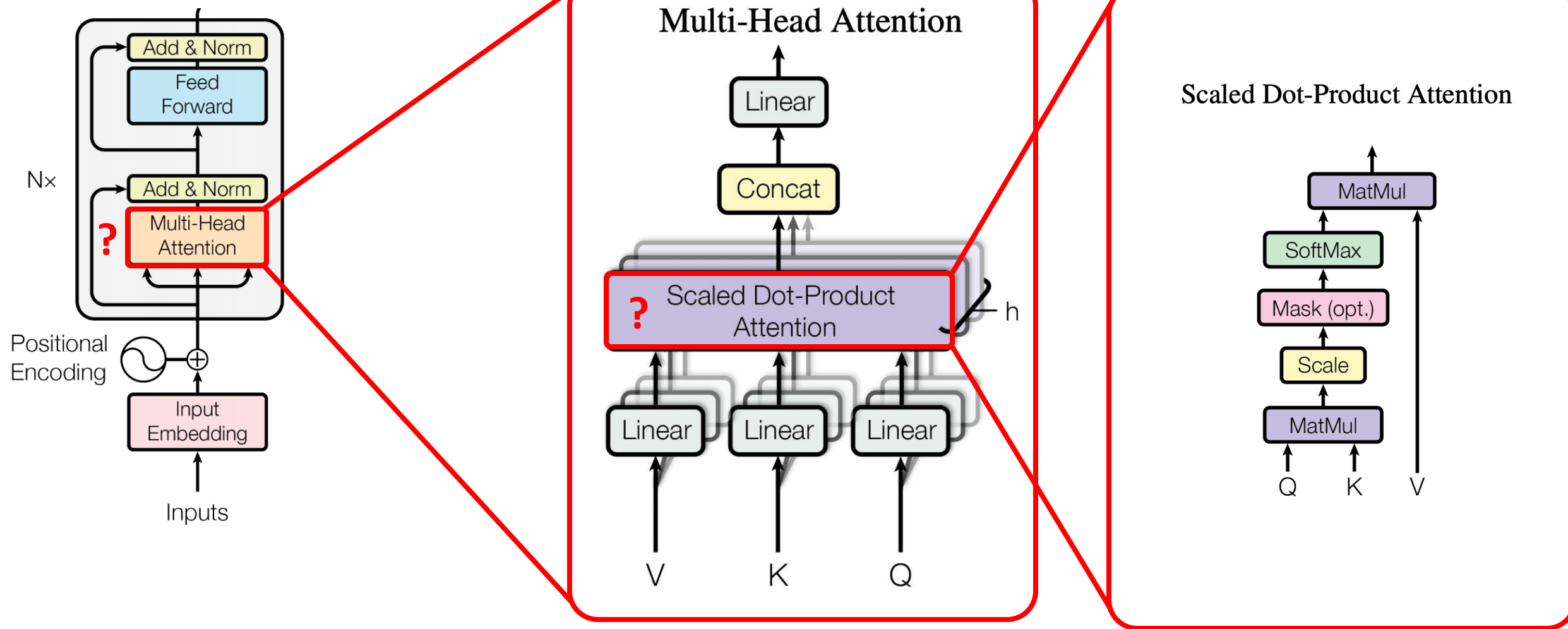




Take a breath.

We'll start digging here.





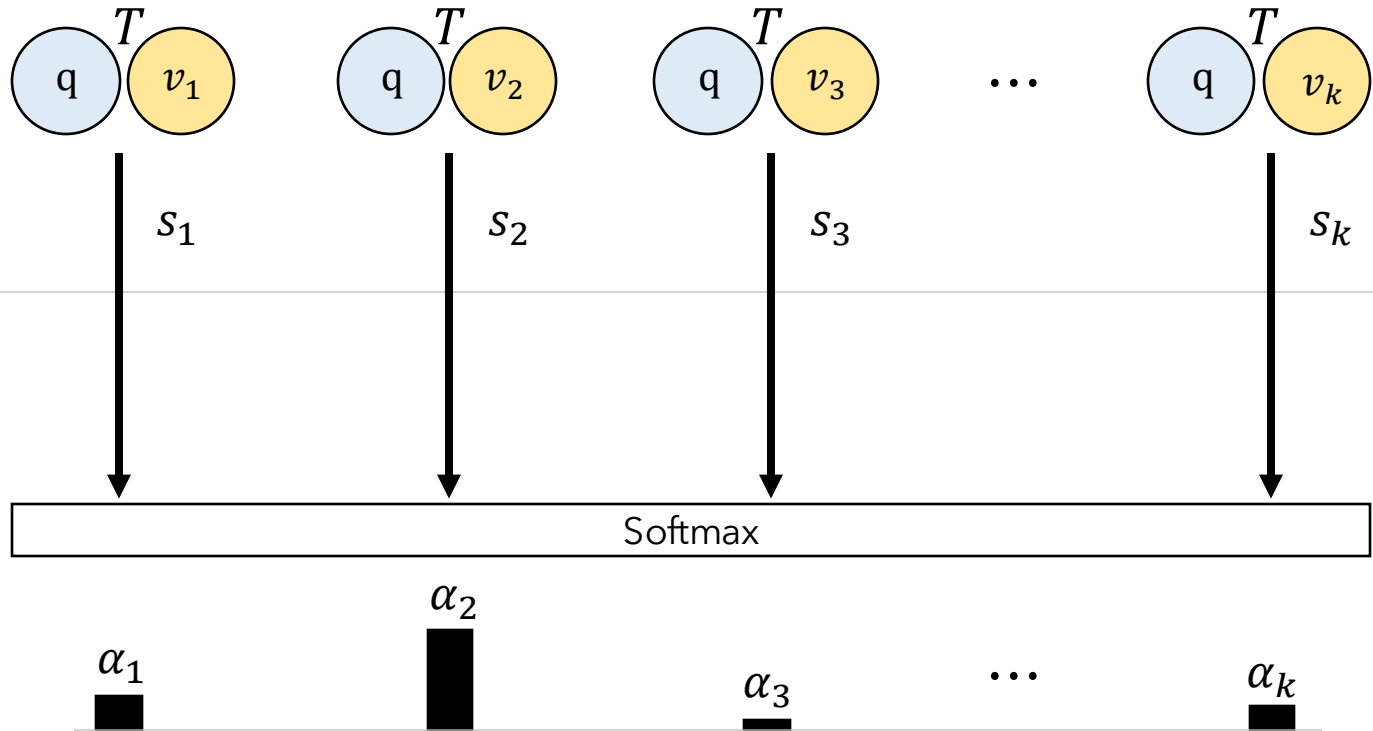


# One Example of A Simple Soft Attention Mechanism

## Step 1:

Compute Some Relevancy Score

$$s_i = q^T v_i$$



## Step 2:

Normalize to Attention Distribution

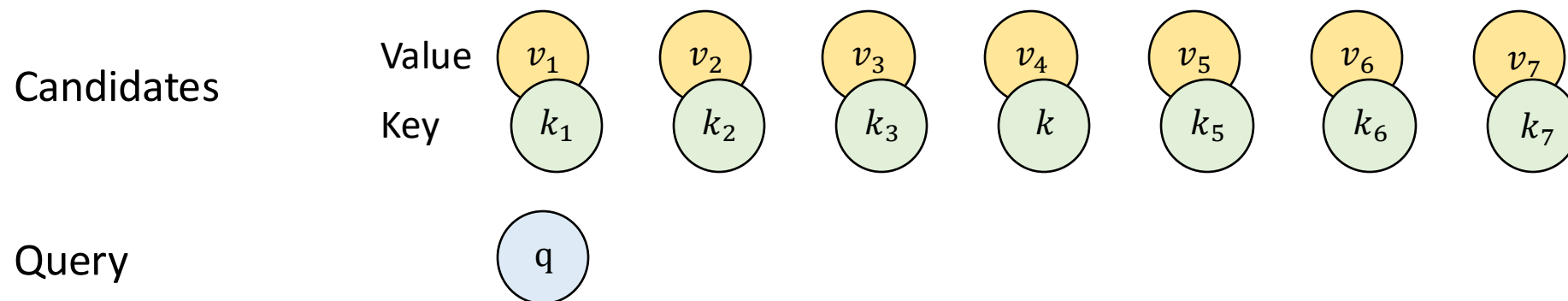
$$\alpha_i = \text{softmax}_i(s_i)$$

## Step 3:

Produce Attended Feature

$$v = \sum_i \alpha_i v_i$$

$$\alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3 + \dots + \alpha_k v_k$$



Attention Function

$$\alpha_i = \text{softmax}_i \left( \frac{q k_i^T}{\sqrt{d_k}} \right)$$

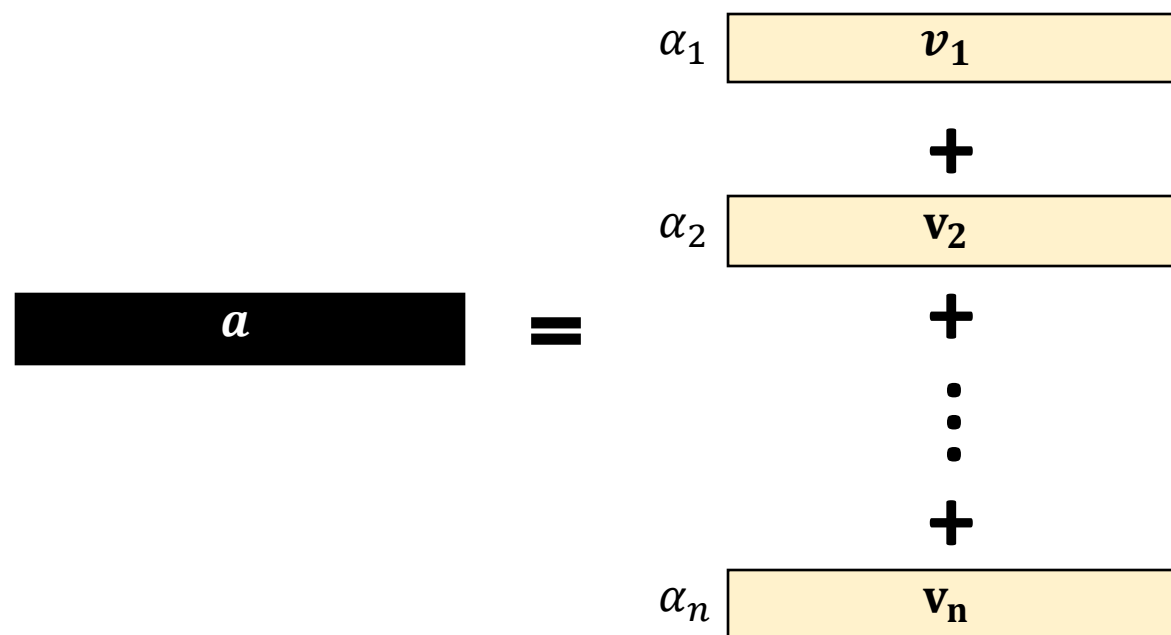
Attended Feature

$$a = \sum_i \alpha_i v_i$$

$$\alpha_i = \text{softmax}_i \left( \frac{qk_i^T}{\sqrt{d_k}} \right)$$

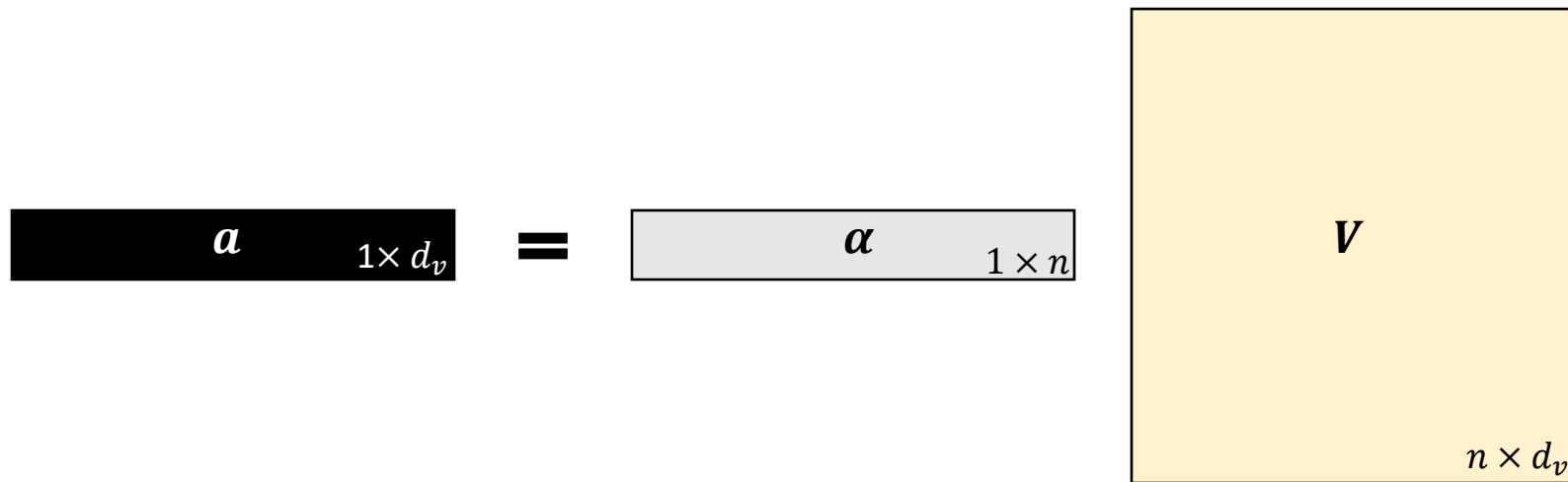
$$\alpha_{1 \times n} = \text{softmax} \left( d_k^{-1/2} \mathbf{q}_{1 \times d_k} \mathbf{K}_{n \times d_k}^T \right)$$

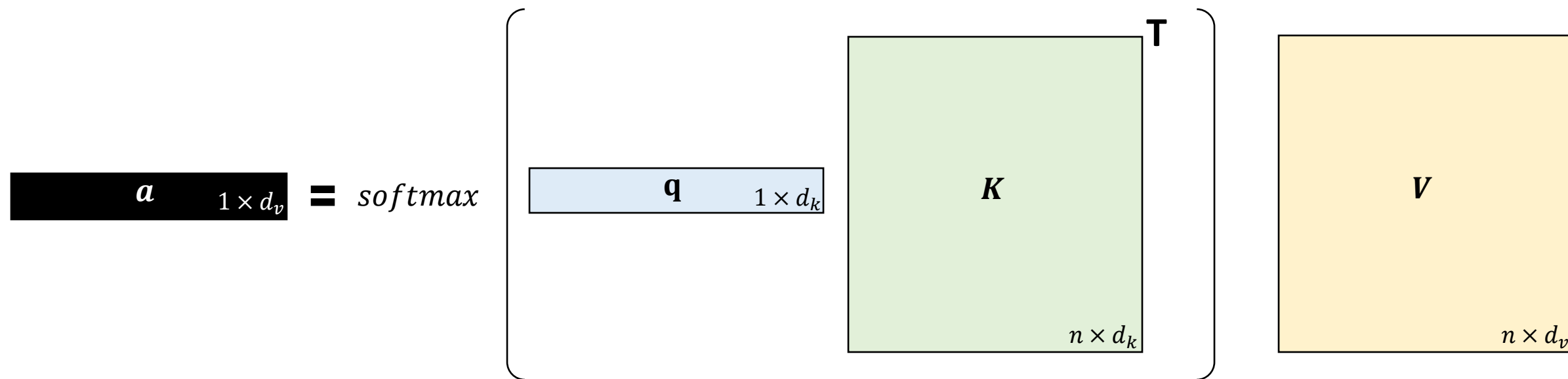
$$a = \sum_i \alpha_i v_i$$



The diagram illustrates the computation of the attention score  $a$ . On the left, a black box labeled  $a$  is followed by an equals sign. To the right of the equals sign is a vertical stack of terms representing the weighted sum. The first term is  $\alpha_1$  followed by a yellow box containing  $v_1$ . This is followed by a plus sign, then  $\alpha_2$  followed by a yellow box containing  $v_2$ . This is followed by another plus sign, then three vertical dots indicating the continuation of the sum, followed by a third plus sign, and finally  $\alpha_n$  followed by a yellow box containing  $v_n$ .

$$a = \sum_i \alpha_i v_i$$





Matrix Form

$$\mathbf{a} = \text{softmax} \left( \frac{\mathbf{q} \mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

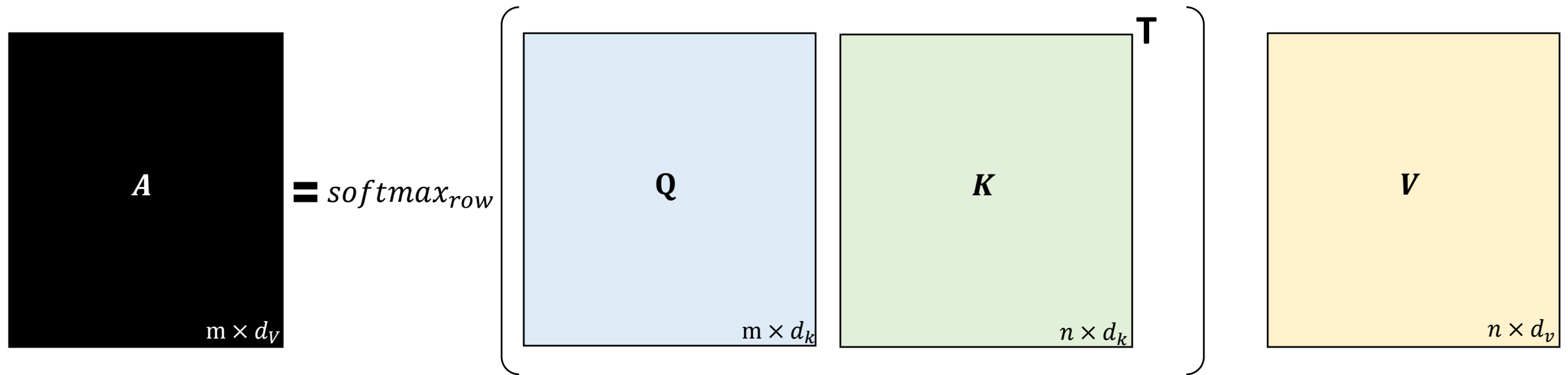




# Scaled-Dot Product Attention

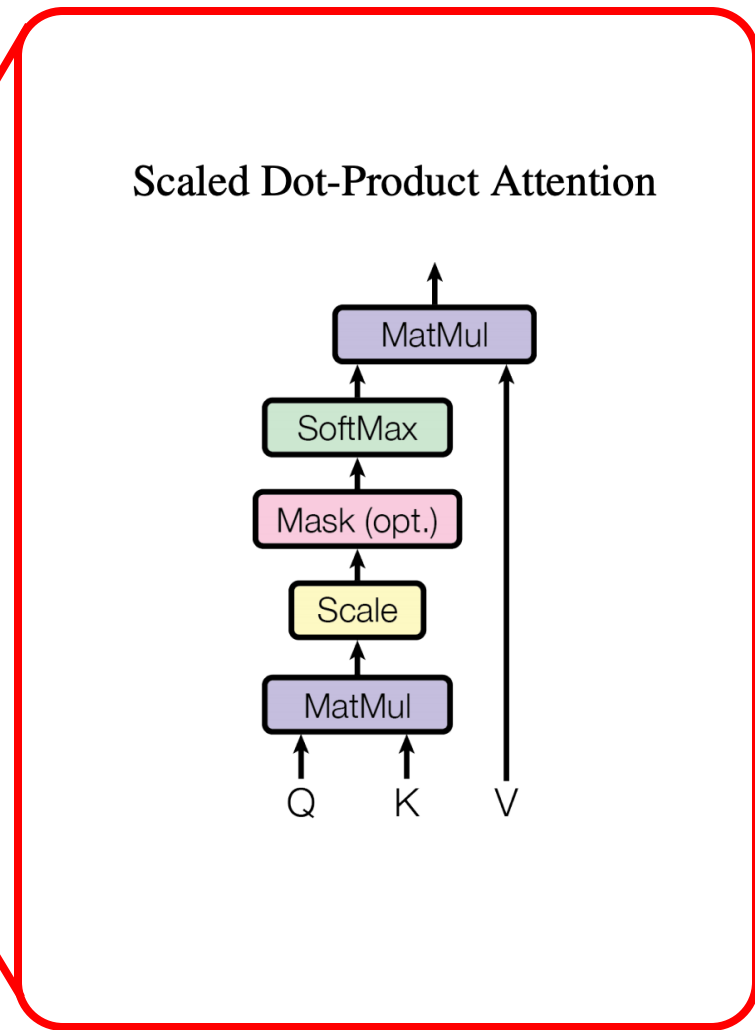
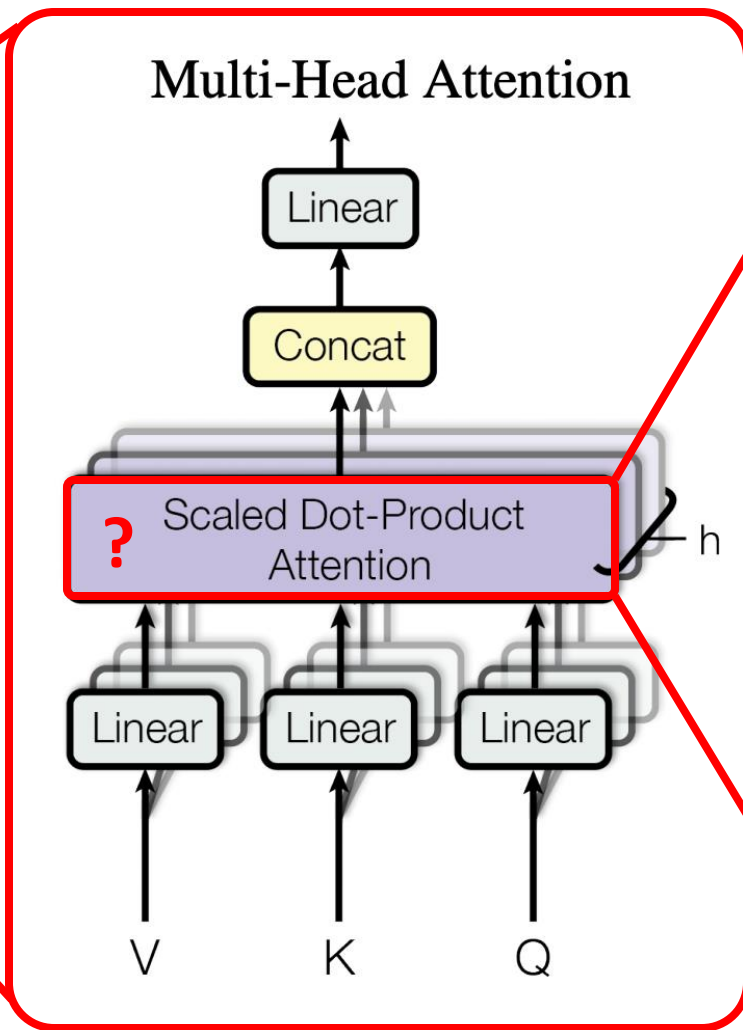
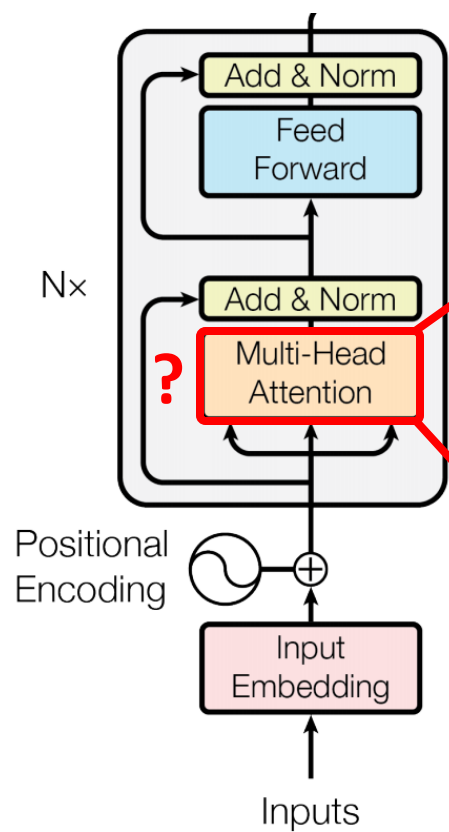
$$\begin{bmatrix} \mathbf{a}_1 & 1 \times d_v \\ \mathbf{a}_2 & 1 \times d_v \end{bmatrix} = \text{softmax}_{\text{row}} \left( \begin{bmatrix} \mathbf{q}_1 & 1 \times d_k \\ \mathbf{q}_2 & 1 \times d_k \end{bmatrix} \begin{bmatrix} K \\ n \times d_k \end{bmatrix}^T \right) \begin{bmatrix} V \\ n \times d_v \end{bmatrix}$$

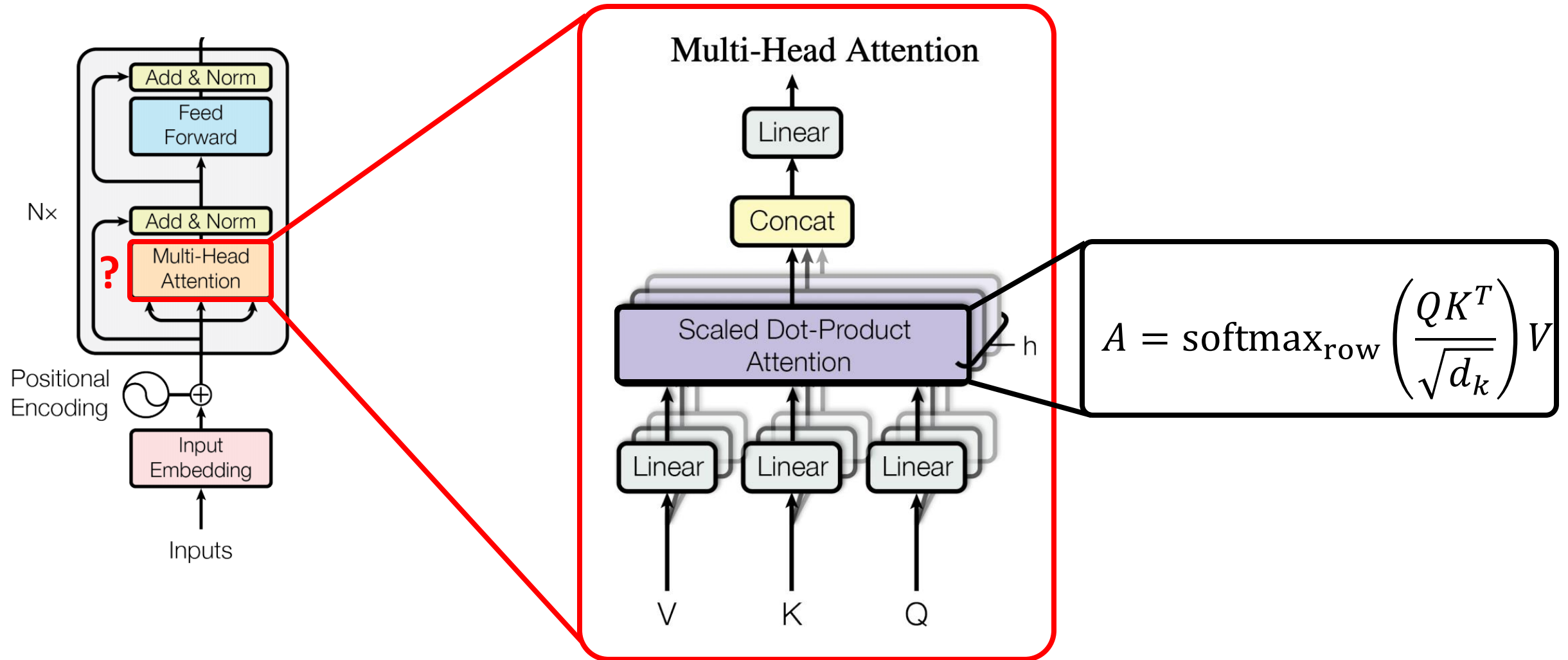
The diagram illustrates the Scaled-Dot Product Attention mechanism. On the left, a matrix of query vectors  $\mathbf{a}_1, \mathbf{a}_2$  with dimensions  $1 \times d_v$  is shown. This matrix is multiplied by the result of a row-wise softmax operation. The softmax operation is applied to the product of a matrix of query vectors  $\mathbf{q}_1, \mathbf{q}_2$  (dimensions  $1 \times d_k$ ) and the transpose of a key matrix  $K$  (dimensions  $n \times d_k$ ). The result of the softmax operation is then multiplied by a value matrix  $V$  (dimensions  $n \times d_v$ ).



Multi-Query  
Matrix Form

$$A = \text{softmax}_{\text{row}} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$







# Multi-Head Attention

$$Q^{(1)} = QW_Q^{(1)}$$

$$V^{(1)} = VW_V^{(1)}$$

$$K^{(1)} = KW_K^{(1)}$$

$$Q^{(2)} = QW_Q^{(2)}$$

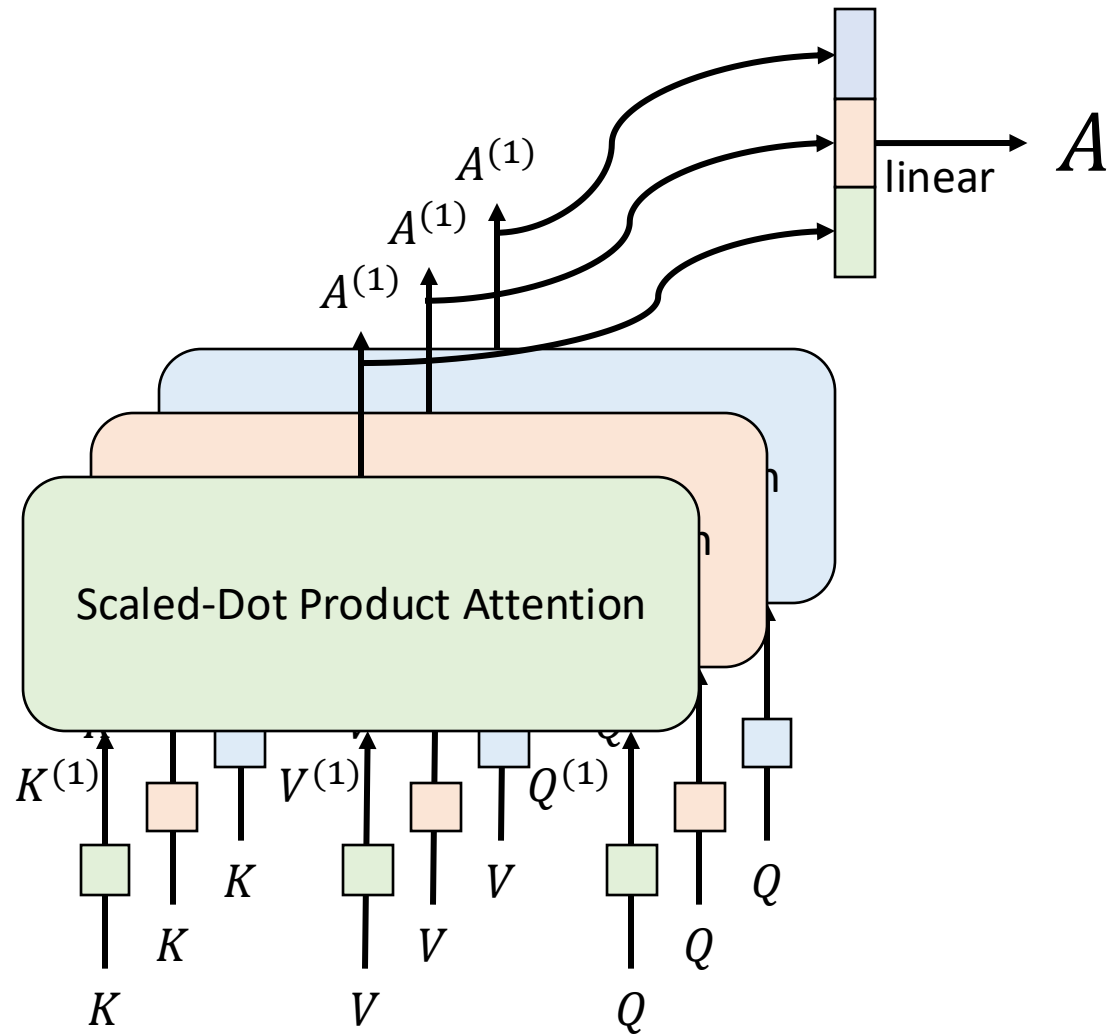
$$V^{(2)} = VW_V^{(2)}$$

$$K^{(2)} = KW_K^{(2)}$$

$$Q^{(3)} = QW_Q^{(3)}$$

$$V^{(3)} = VW_V^{(3)}$$

$$K^{(3)} = KW_K^{(3)}$$





1. Take  $H$  projections of the query, value, key matrices.

$$Q^{(h)} = QW_Q^{(h)} \quad V^{(h)} = VW_V^{(h)} \quad K^{(h)} = KW_K^{(h)}$$

2. Perform full multi-query Scaled-Dot Product Attention (call this an attn. head)

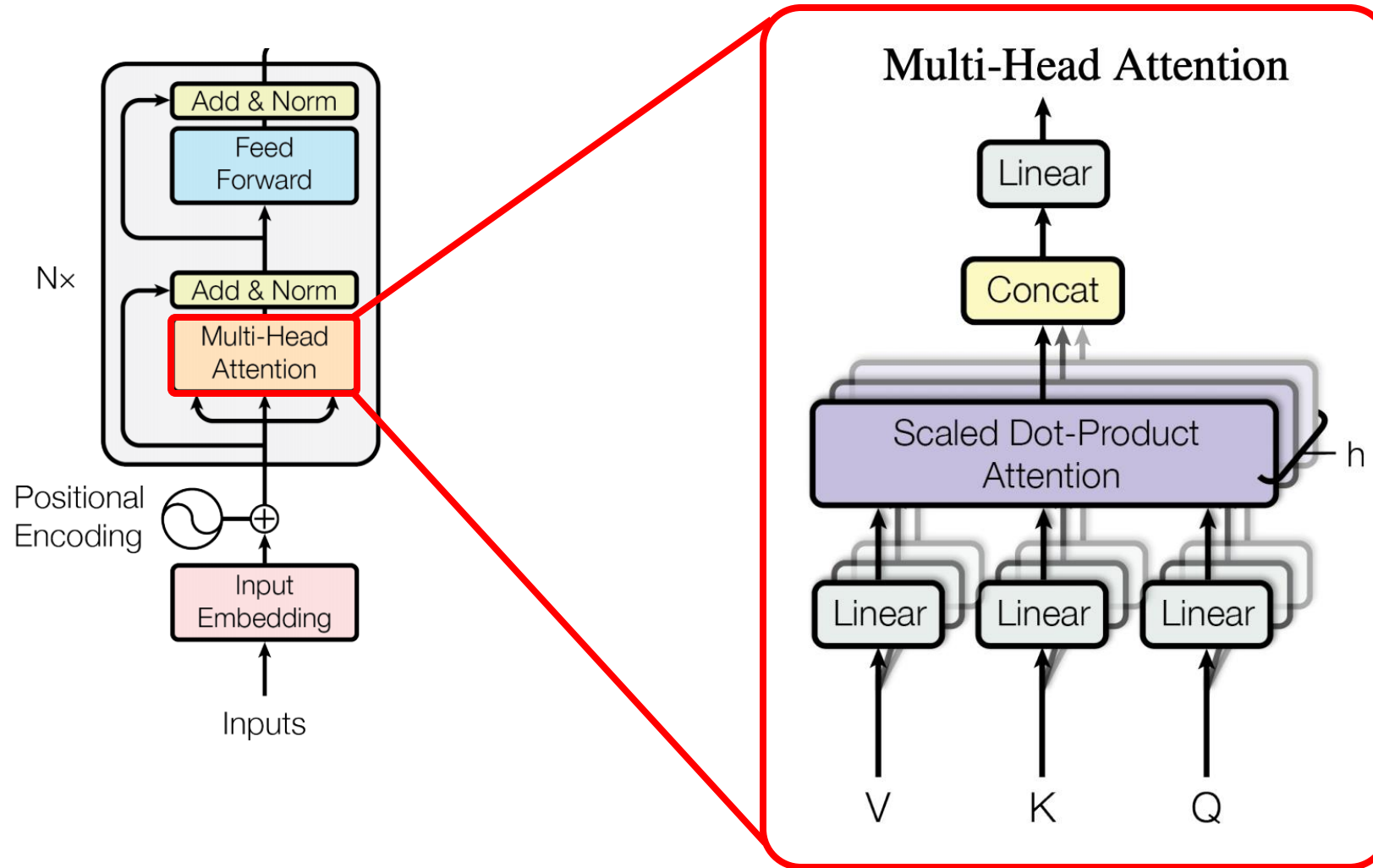
$$A^{(h)} = \text{softmax}_{\text{row}} \left( \frac{Q^{(h)} K^{(h)T}}{\sqrt{d_k}} \right) V^{(h)}$$

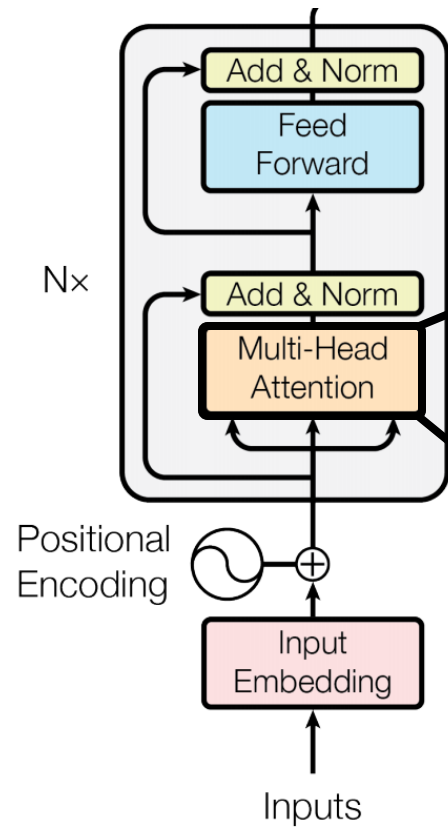
3. Combine the output from all attention heads

$$A_{i*} = \text{concat}(A_{i*}^1, \dots, A_{i*}^h) W_O$$

$1 \times Hd_v \quad Hd_v \times d_X$







1. Take  $\mathbf{H}$  projections of the query, value, key matrices.

$$Q^{(h)} = QW_Q^{(h)} \quad V^{(h)} = VW_V^{(h)} \quad K^{(h)} = KW_K^{(h)}$$

2. Perform full multi-query Scaled-Dot Product Attention (call this an attn. head)

$$A^{(h)} = \text{softmax}_{\text{row}} \left( \frac{Q^{(h)} K^{(h)T}}{\sqrt{d_k}} \right) V^{(h)}$$

3. Combine the output from all attention heads

$$A_{i*} = \text{concat}(A_{i*}^1, \dots, A_{i*}^h) W_O$$

$1 \times Hd_v \quad Hd_v \times d_x$



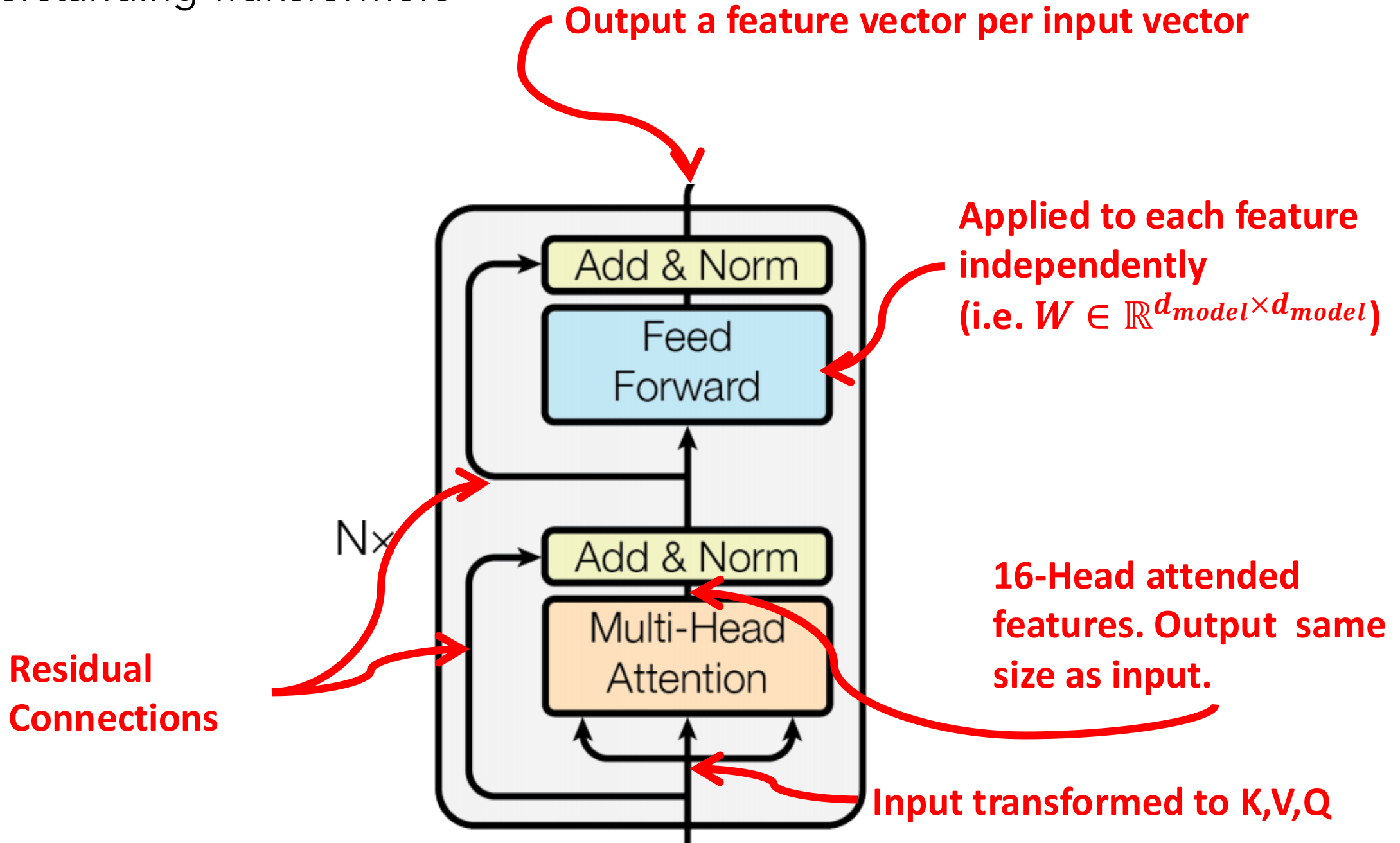
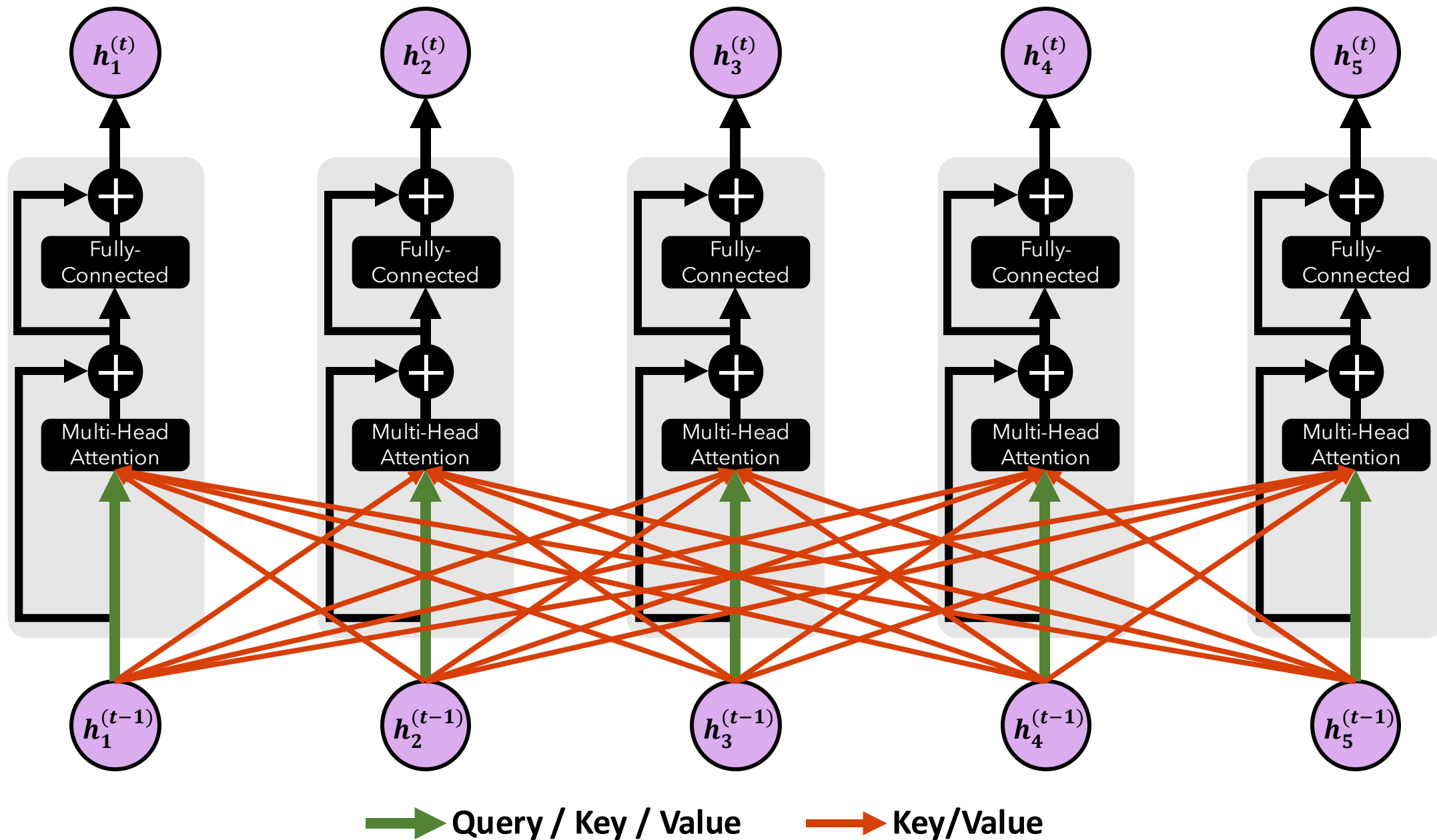


Image Credit: Vaswani et al, 2017.

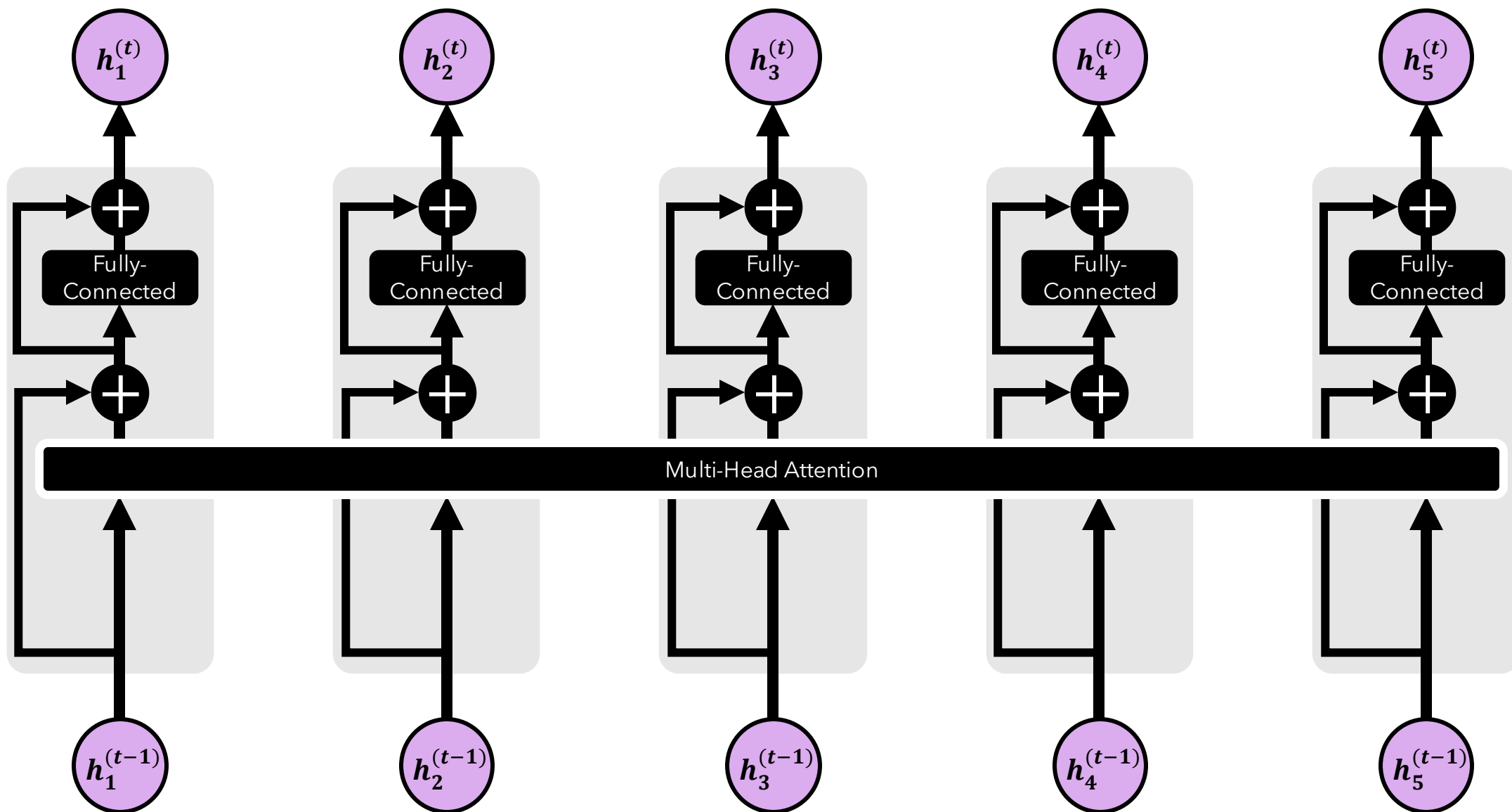


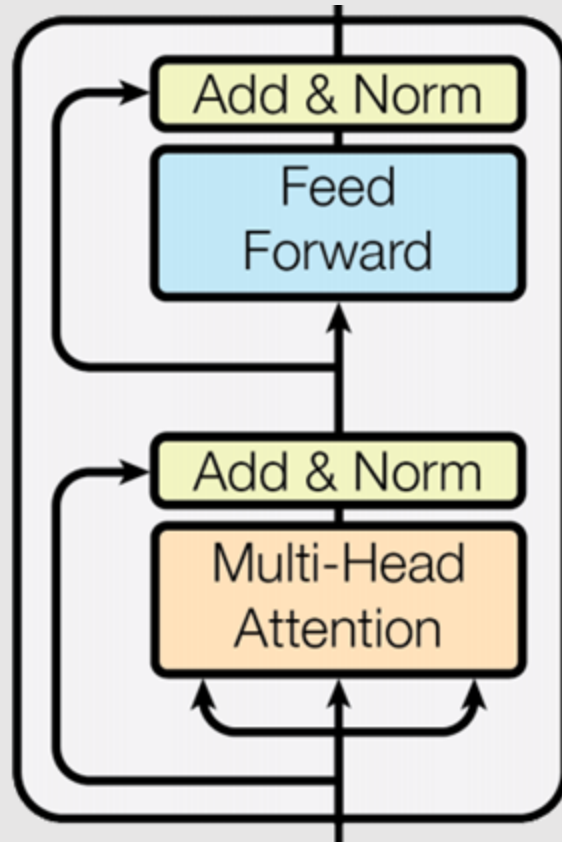
# Transformer Encoder Block From Another View





# Transformer Encoder Block From Another View





1. Compute mean and standard deviation of activations for an instance:

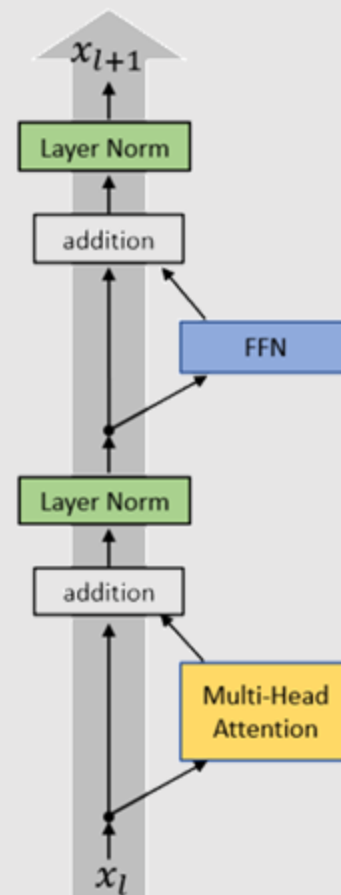
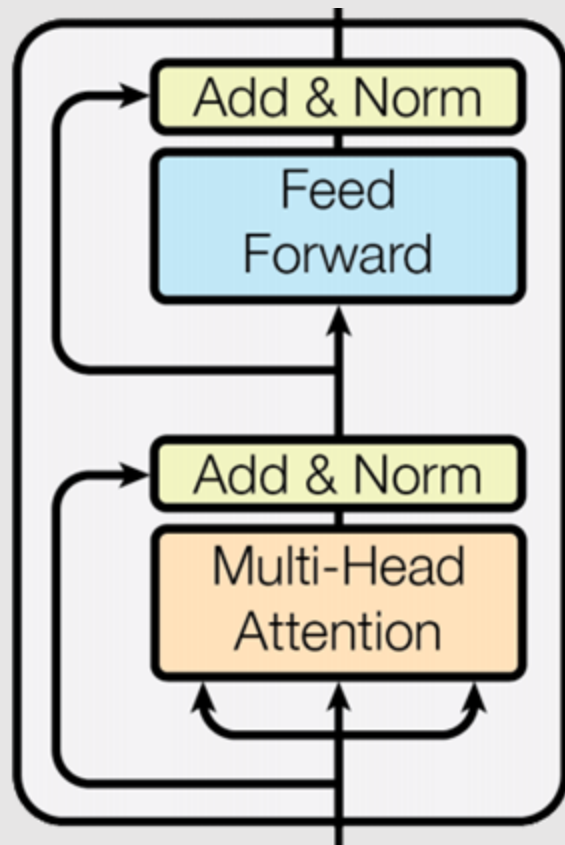
$$\mu = \frac{1}{H} \sum_{i=1}^H a_i$$

$$\sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i - \mu)^2}$$

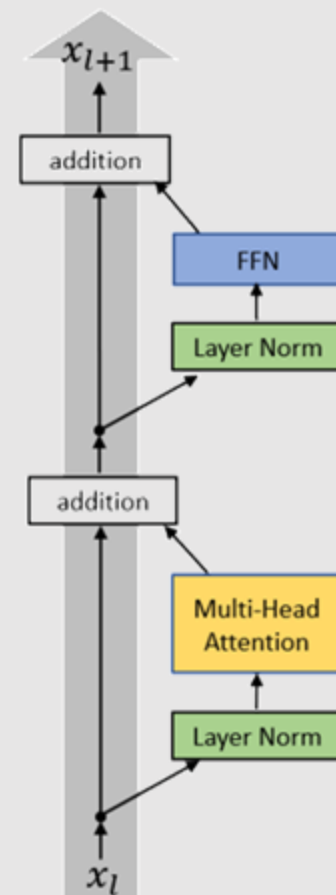
2. Subtract mean and divide by standard deviation.



# An Aside: Layer Normalization



Vanilla (Post-LN)  
Transformer

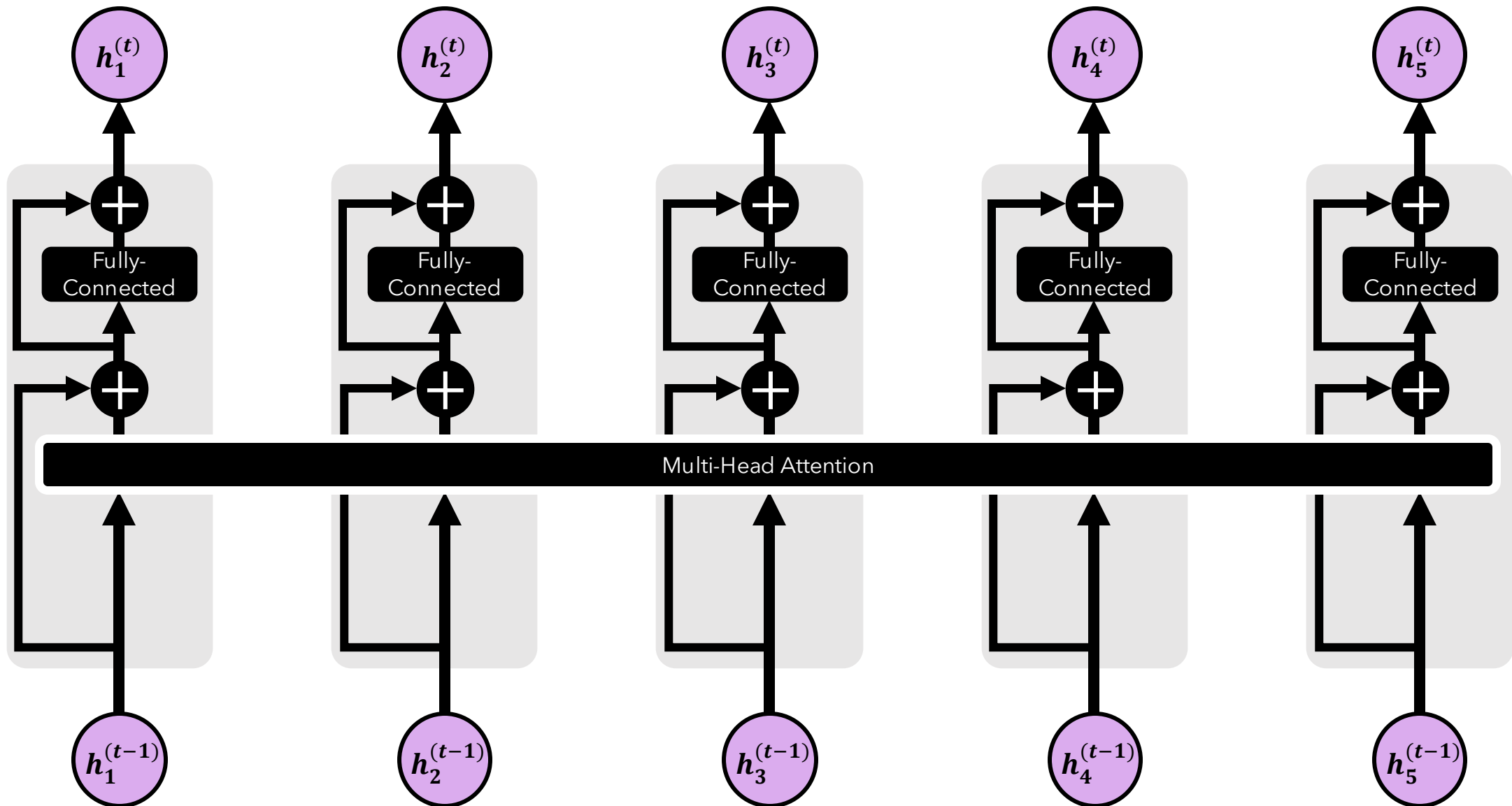


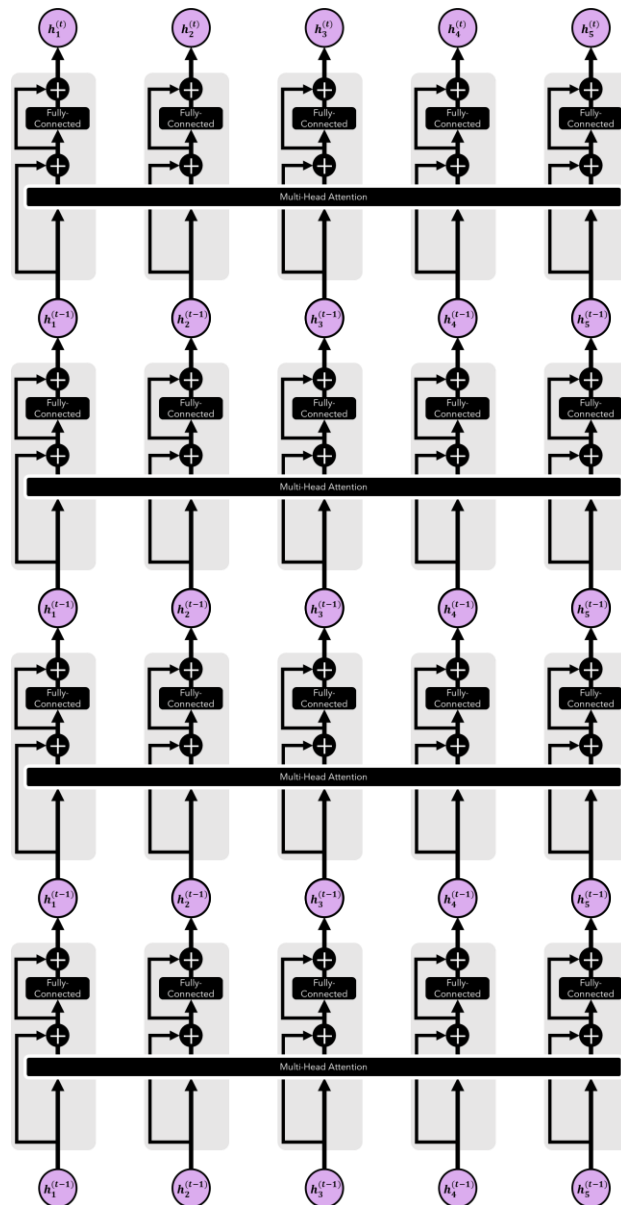
Pre-LN  
Transformer

Switching around the order of add vs. layer norm leads to more stable initial gradients.



# Transformer Encoder Block From Another View





## Can stack multiple transformer layers

- At later layers, the attention mechanism is attending over attended features from the previous layer's attention mechanism....





**Positional Encoding:** A feature representation of position in a sequence

**Want some encoder  $\phi(t) \in \mathbb{R}^d$  that maps position  $t$  to a fixed-size feature:**

- **Uniquely** identifies each position that is likely to be observed
- **Bounded** scale (e.g.  $[-1, 1]$ )
- **Easy** to reason computationally about time **deltas** (e.g. easily identify if two encodings are for positions some fixed distance apart)
- **Generalizes** well to longer sequences



**Idea 1: Position.** Just use integers 1,2,....

**Idea 2: Normalized Position.** Just use position / length

**Idea 3: Embedding.** Learn an embedding for  $t=1, \dots, \text{max\_length}$

**Idea 4: Periodic.** Pass  $t$  through a battery of periodic functions

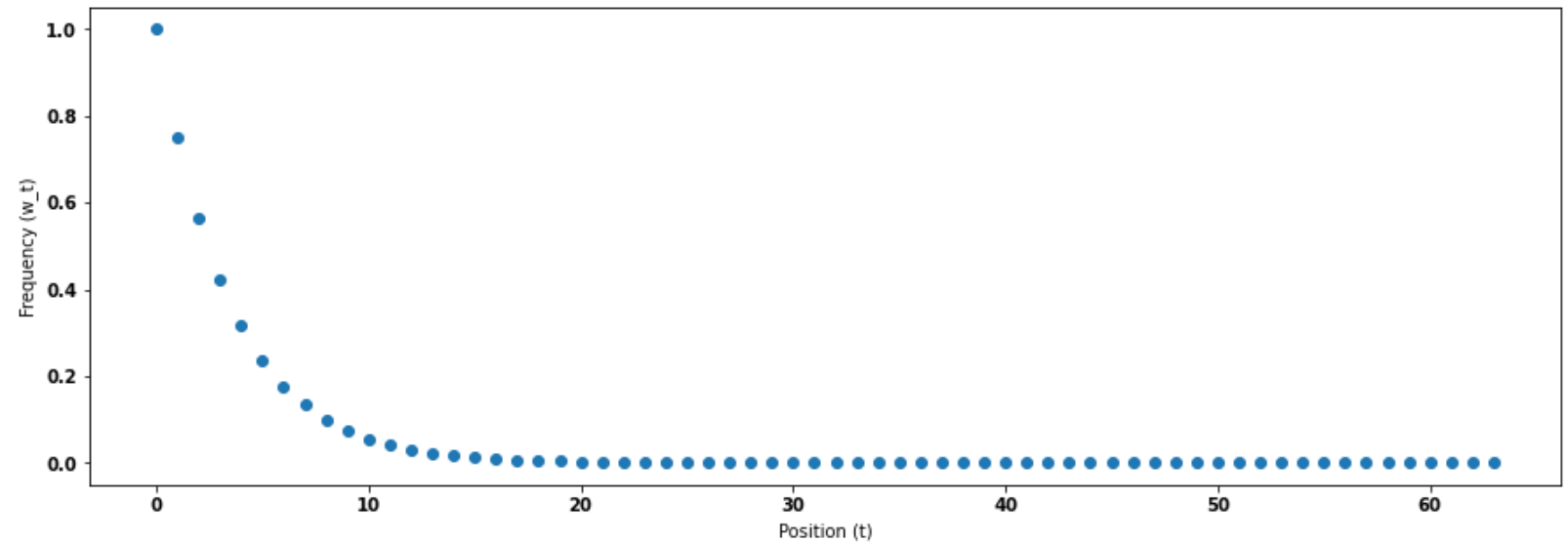
|             | Idea 1:<br>Position | Idea 2:<br>Normalized<br>Position | Idea 3:<br>Learn<br>Embedding | Idea 4:<br>Periodic<br>Encoding |
|-------------|---------------------|-----------------------------------|-------------------------------|---------------------------------|
| Unique      | ✓                   | ✓                                 | ✓                             | ✓                               |
| Bounded     | ✗                   | ✓                                 | ✓                             | ✓                               |
| Easy delta  | ✓                   | ✗                                 | ✓                             | ✓                               |
| Generalizes | ✗                   | ✗                                 | ✗                             | ✗                               |

**VOCAB**

**Periodic Position Encoding:** A vector of periodic functions at different frequencies representing a scalar input (usually position)

$$\phi(t) = \begin{bmatrix} \sin(w_1 * t) \\ \cos(w_1 * t) \\ \sin(w_2 * t) \\ \cos(w_2 * t) \\ \vdots \\ \cos(w_d * t) \\ \cos(w_d * t) \end{bmatrix}$$

$$w_i = \frac{1}{10000^{(2i/d)}}$$



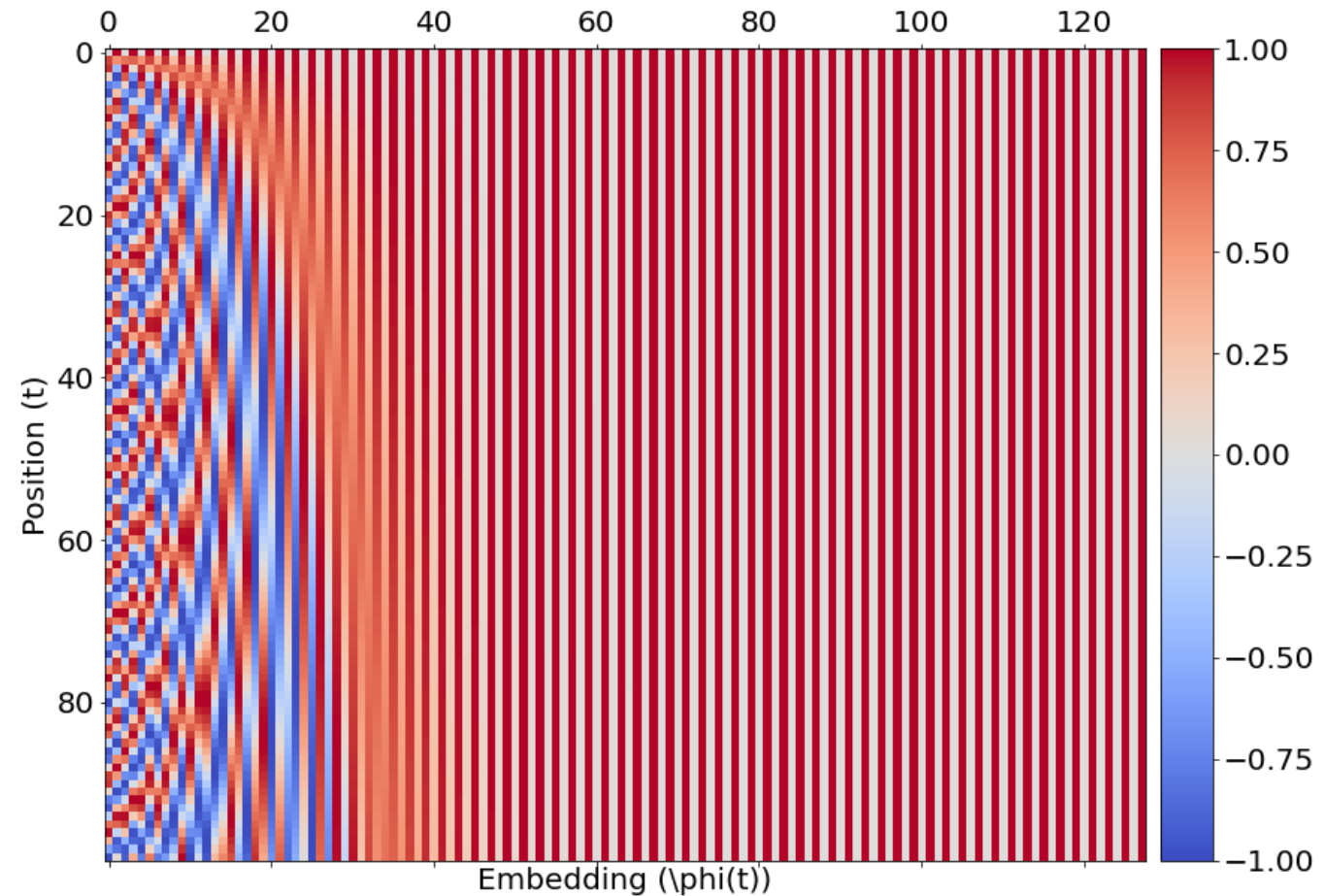
**Bounded:** Cosine/ Sin are bounded functions.

**VOCAB**

**Periodic Position Encoding:** A vector of periodic functions at different frequencies representing a scalar input (usually position)

$$\phi(t) = \begin{bmatrix} \sin(w_1 * t) \\ \cos(w_1 * t) \\ \sin(w_2 * t) \\ \cos(w_2 * t) \\ \vdots \\ \cos(w_d * t) \\ \cos(w_d * t) \end{bmatrix}$$

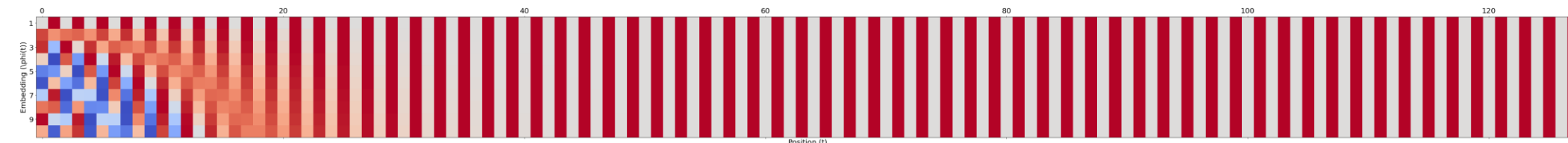
$$w_i = \frac{1}{10000^{(2i/d)}}$$



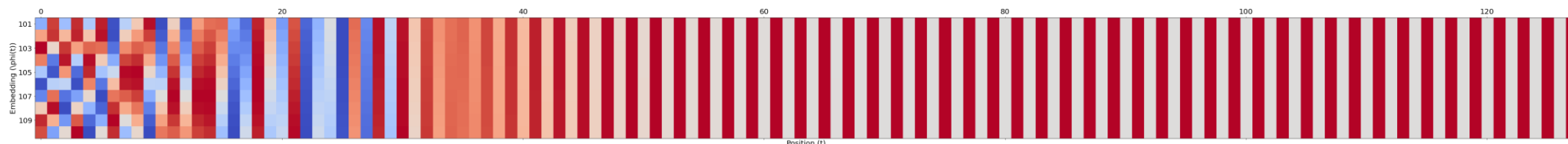


# Periodic Position Encodings

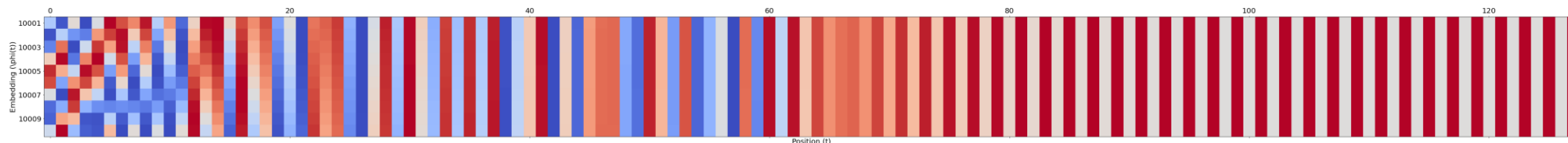
0 -  
10



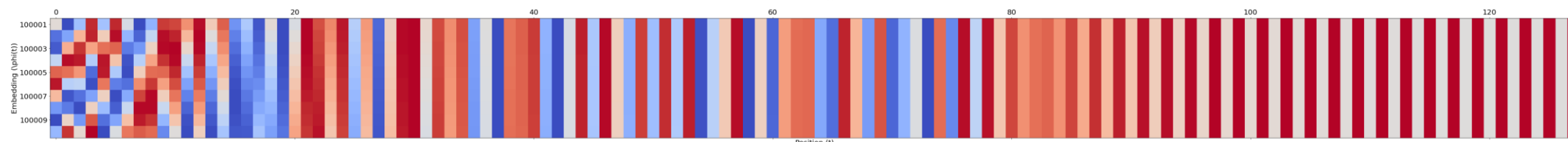
100 -  
110



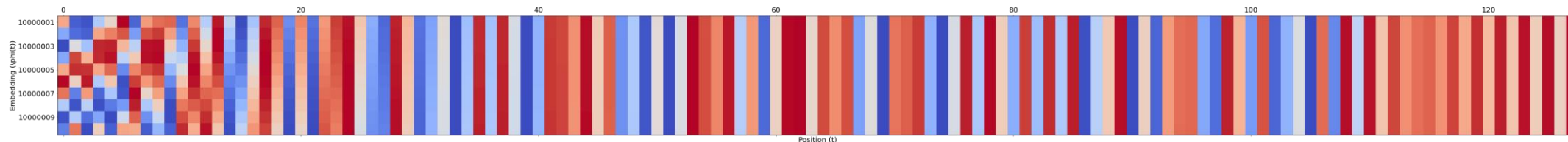
10000 -  
10010



100000 -  
100010



10000000 -  
10000010

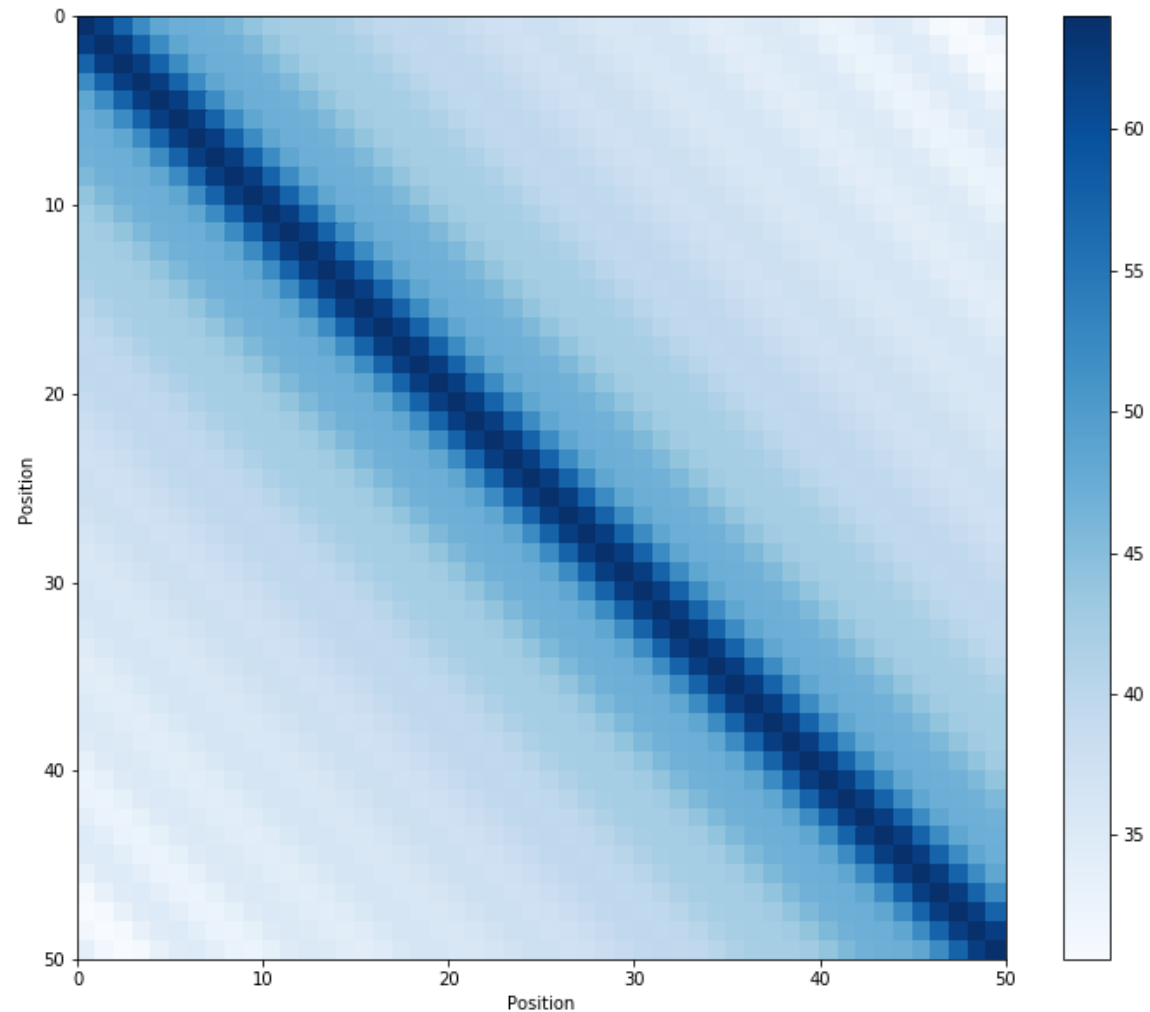


**Uniqueness:** Will eventually circle around – but takes so very long that it isn't a real concern.





# Periodic Position Encodings



**Easy delta:** Dot-product distance between position encoding decays symmetrically.





Let  $d = 1$  for simplicity, such that  $\phi(t) = [\sin(w_1 t); \cos(w_1 t)]$ :

**Claim:**  $\exists M_k \quad s.t. \quad M_k \phi(t) = \phi(t + k) \quad \forall k$

$$\begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} \sin(w_1 t) \\ \cos(w_1 t) \end{bmatrix} = \begin{bmatrix} \sin(w_1(t + k)) \\ \cos(w_1(t + k)) \end{bmatrix}$$

Angle Sum  
Identity:

$$\begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} \sin(w_1 t) \\ \cos(w_1 t) \end{bmatrix} = \begin{bmatrix} \cos(w_1 k) \sin(w_1 t) + \sin(w_1 k) \cos(w_1 t) \\ \sin(w_1 k) \sin(w_1 t) + \cos(w_1 k) \cos(w_1 t) \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{m_{11}} \sin(w_1 t) + \mathbf{m_{12}} \cos(w_1 t) \\ \mathbf{m_{21}} \sin(w_1 t) + \mathbf{m_{22}} \cos(w_1 t) \end{bmatrix} = \begin{bmatrix} \mathbf{\cos(w_1 k)} \sin(w_1 t) + \mathbf{\sin(w_1 k)} \cos(w_1 t) \\ \mathbf{\sin(w_1 k)} \sin(w_1 t) + \mathbf{\cos(w_1 k)} \cos(w_1 t) \end{bmatrix}$$

$$M_k = \begin{bmatrix} \cos(w_1 k) & \sin(w_1 k) \\ \sin(w_1 k) & \cos(w_1 k) \end{bmatrix} \quad \blacksquare$$

**Easy delta:** Linear transform between representations of fixed temporal distance.





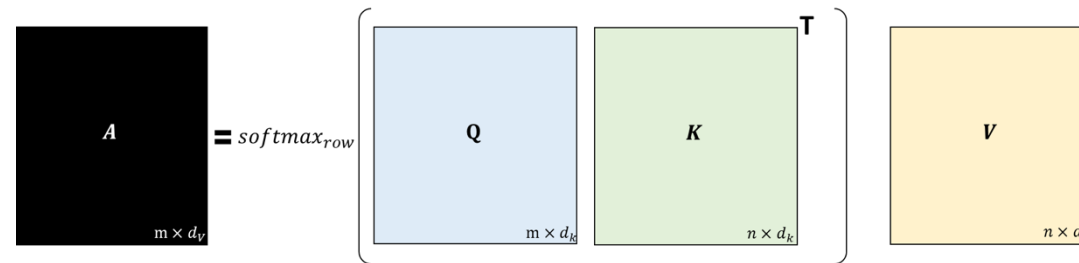
# Periodic Position Encodings

Let  $d = 1$  for simplicity, such that  $\phi(t) = [\sin(w_1 * t); \cos(w_1 + t)]:$

$$\text{Claim: } \exists M \text{ s.t. } M \phi(t) = \phi(t + l) \quad \forall l$$

## Neat math but why do we care?

Keys and Queries for attention are linear transforms of representations. Makes it easy to make an attention head look for specific relative positions (e.g., previous word)



$$K = W_k h_1^{(t)}$$

$$Q = W_Q h_1^{(t)}$$

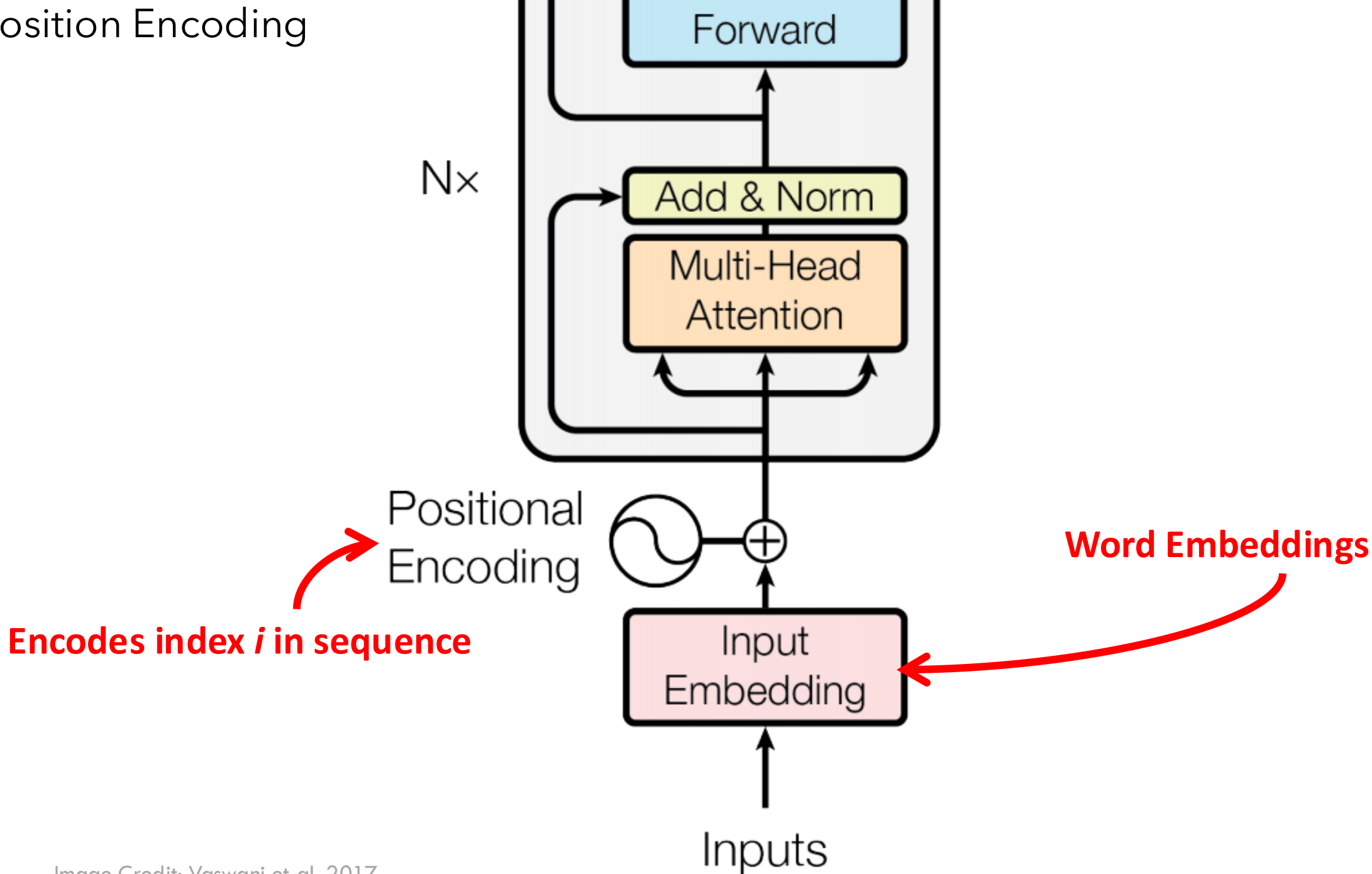
Multi-Query Matrix Form

$$A = \text{softmax}_{\text{row}} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

$$M_k = \begin{bmatrix} \cos(w_1 * k) & \sin(w_1 * k) \\ \sin(w_1 * k) & \cos(w_1 * k) \end{bmatrix}$$

**Easy delta:** Linear transform between representations of fixed temporal distance.







# Encoder-Decoder Model for Translation

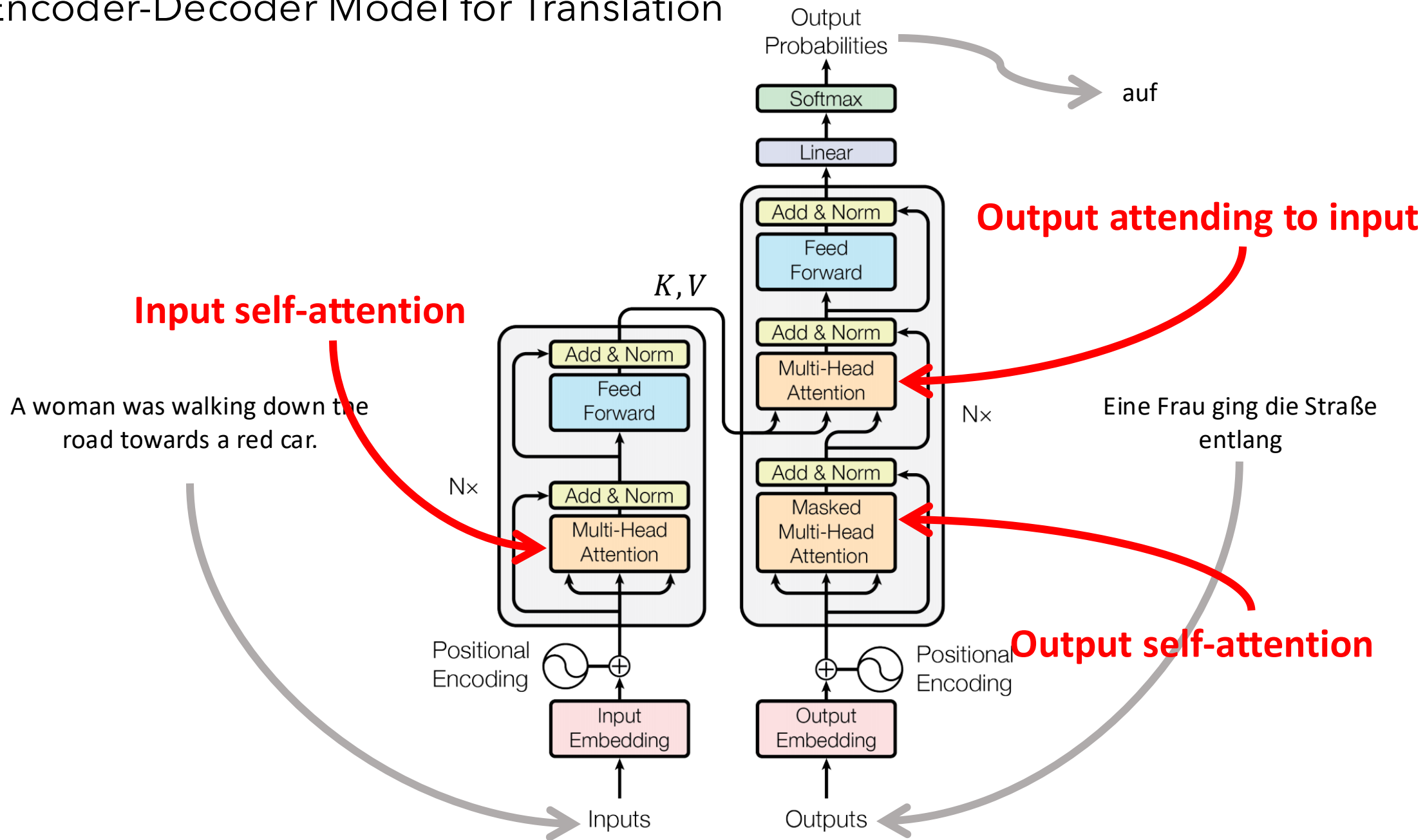


Image Credit: Vaswani et al, 2017.



# Some Results from Transformer Paper: Machine Translation

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

## Attention Is All You Need

Ashish Vaswani\*  
Google Brain  
avaswani@google.com

Noam Shazeer\*  
Google Brain  
noam@google.com

Niki Parmar\*  
Google Research  
nikip@google.com

Jakob Uszkoreit\*  
Google Research  
usz@google.com

Llion Jones\*  
Google Research  
llion@google.com

Aidan N. Gomez\*<sup>1</sup>  
University of Toronto  
aidan@cs.toronto.edu

Lukas Kaiser\*  
Google Brain  
lukaszkaizer@google.com

Illia Polosukhin\*<sup>2</sup>  
illia.polosukhin@gmail.com

### Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

### 1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and

\*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

<sup>1</sup>Work performed while at Google Brain.

<sup>2</sup>Work performed while at Google Research.

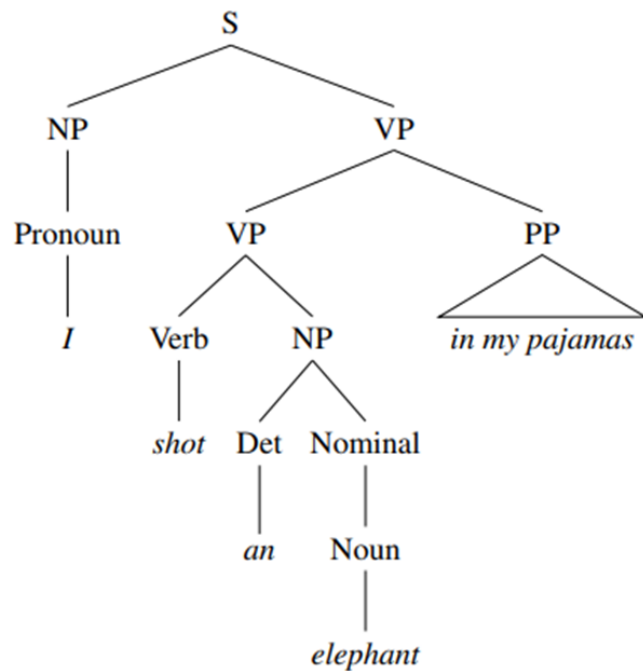
| Model                           | BLEU        |              | Training Cost (FLOPs)                 |                     |
|---------------------------------|-------------|--------------|---------------------------------------|---------------------|
|                                 | EN-DE       | EN-FR        | EN-DE                                 | EN-FR               |
| ByteNet [18]                    | 23.75       |              |                                       |                     |
| Deep-Att + PosUnk [39]          |             | 39.2         |                                       | $1.0 \cdot 10^{20}$ |
| GNMT + RL [38]                  | 24.6        | 39.92        | $2.3 \cdot 10^{19}$                   | $1.4 \cdot 10^{20}$ |
| ConvS2S [9]                     | 25.16       | 40.46        | $9.6 \cdot 10^{18}$                   | $1.5 \cdot 10^{20}$ |
| MoE [32]                        | 26.03       | 40.56        | $2.0 \cdot 10^{19}$                   | $1.2 \cdot 10^{20}$ |
| Deep-Att + PosUnk Ensemble [39] |             | 40.4         |                                       | $8.0 \cdot 10^{20}$ |
| GNMT + RL Ensemble [38]         | 26.30       | 41.16        | $1.8 \cdot 10^{20}$                   | $1.1 \cdot 10^{21}$ |
| ConvS2S Ensemble [9]            | 26.36       | <b>41.29</b> | $7.7 \cdot 10^{19}$                   | $1.2 \cdot 10^{21}$ |
| Transformer (base model)        | 27.3        | 38.1         | <b><math>3.3 \cdot 10^{18}</math></b> |                     |
| Transformer (big)               | <b>28.4</b> | <b>41.8</b>  | $2.3 \cdot 10^{19}$                   |                     |

1-3 point improvements in BLEU over prior models while requiring less compute.

Mostly exciting because it did so without recurrence!



# Some Results from Transformer Paper: Constituency Parsing

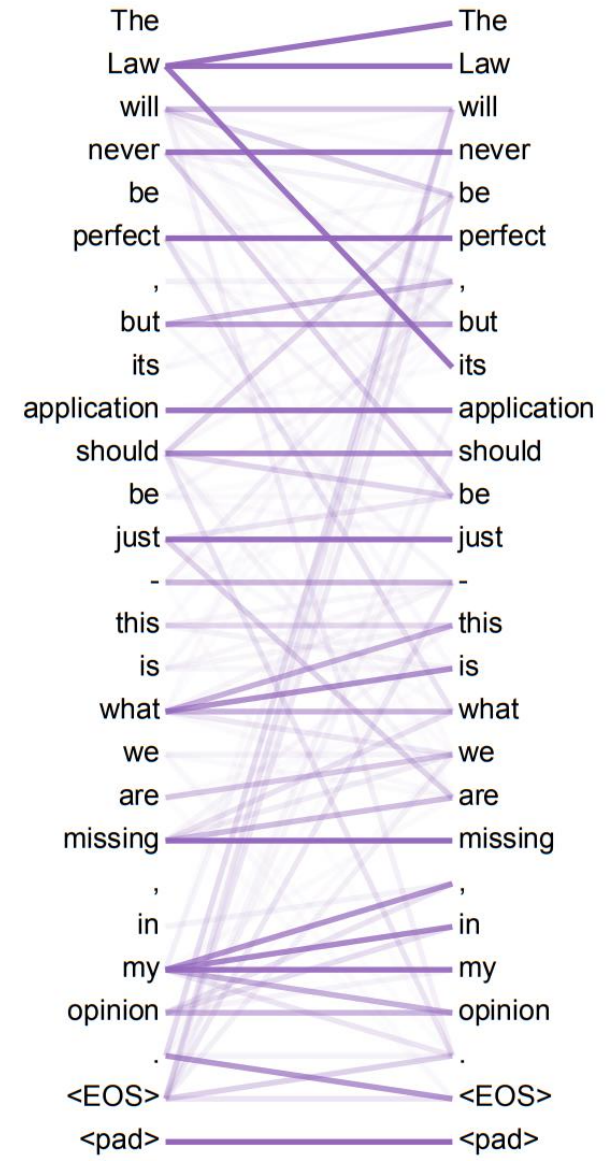
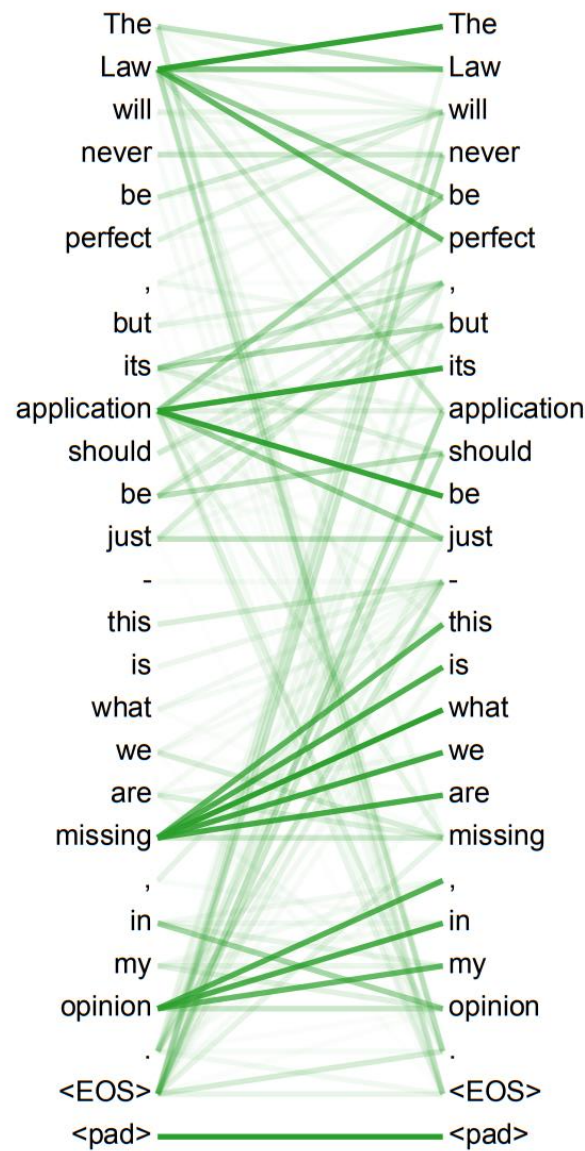
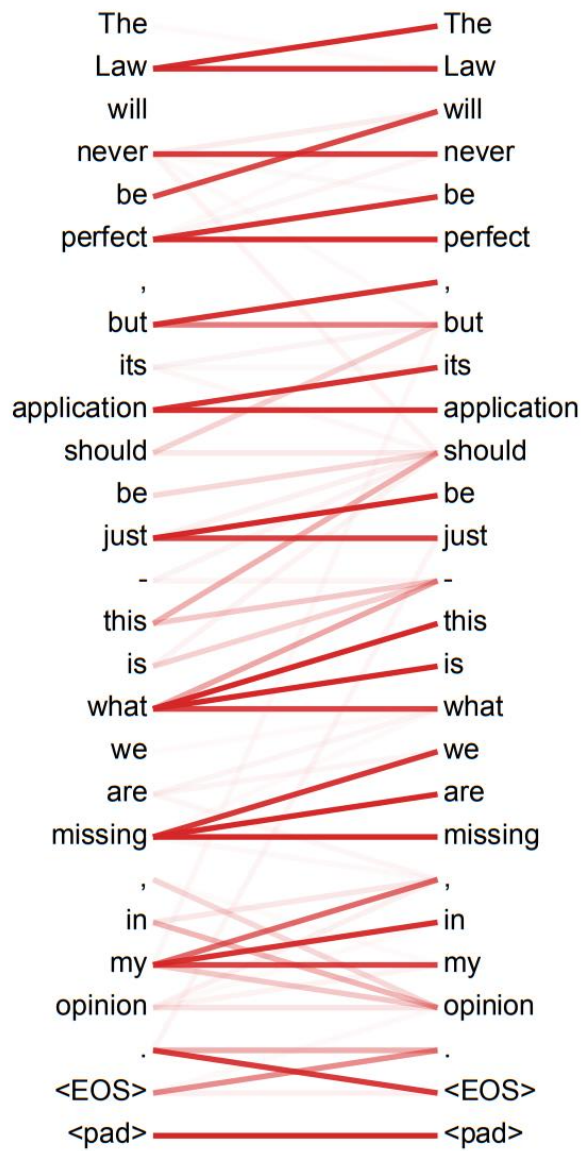


| Parser                              | Training                 | WSJ 23 F1 |
|-------------------------------------|--------------------------|-----------|
| Vinyals & Kaiser et al. (2014) [37] | WSJ only, discriminative | 88.3      |
| Petrov et al. (2006) [29]           | WSJ only, discriminative | 90.4      |
| Zhu et al. (2013) [40]              | WSJ only, discriminative | 90.4      |
| Dyer et al. (2016) [8]              | WSJ only, discriminative | 91.7      |
| Transformer (4 layers)              | WSJ only, discriminative | 91.3      |
| Zhu et al. (2013) [40]              | semi-supervised          | 91.3      |
| Huang & Harper (2009) [14]          | semi-supervised          | 91.3      |
| McClosky et al. (2006) [26]         | semi-supervised          | 92.1      |
| Vinyals & Kaiser et al. (2014) [37] | semi-supervised          | 92.1      |
| Transformer (4 layers)              | semi-supervised          | 92.7      |
| Luong et al. (2015) [23]            | multi-task               | 93.0      |
| Dyer et al. (2016) [8]              | generative               | 93.3      |

**Met or beat prior work – again, interesting due to lack of recurrence!**



# Some Results from Transformer Paper: Visualizing Attention



**Different attention heads in the same layer attend in different ways.**

# Today's Learning Objectives

Be able to answer:

- ~~What is an attention mechanism?~~
- ~~What is a transformer architecture?~~
  - ~~How does scaled-dot-product attention work?~~
  - ~~What are positional encodings?~~
- What are the shortcomings of transformers?
  - What directions are being pursued for efficient transformer models?







- **They generally perform well.**
  - Even better with some large-scale pre-training (next lecture).
- **No waiting around for sequential processing during training.**
  - With proper masking in attention (e.g. can't look at future outputs), whole sequences can be trained in a single forward-pass.
- **Long-range dependencies are easy to capture.**
  - Each input is connected to each other input directly through attention.



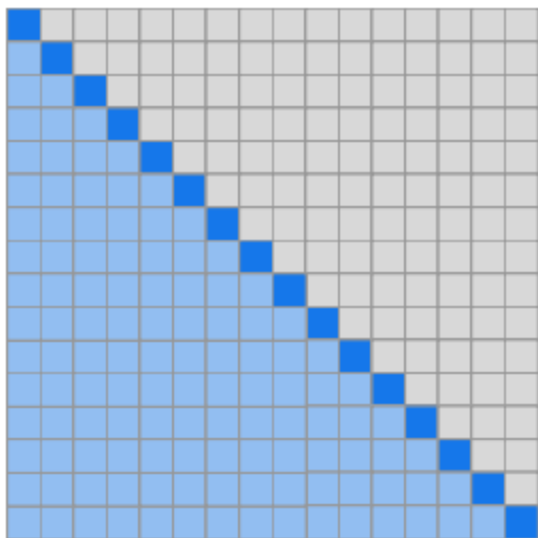
# Why Not To Use Transformers?

- **Their computational cost scales with the square of sequence length due to attention mechanism being all-to-all.**
  - Rough for long sequences. Area of significant research activity.
- **If being used as an online observation encoder, need to recompute things**
  - Or need to be okay with backwards-only attention.

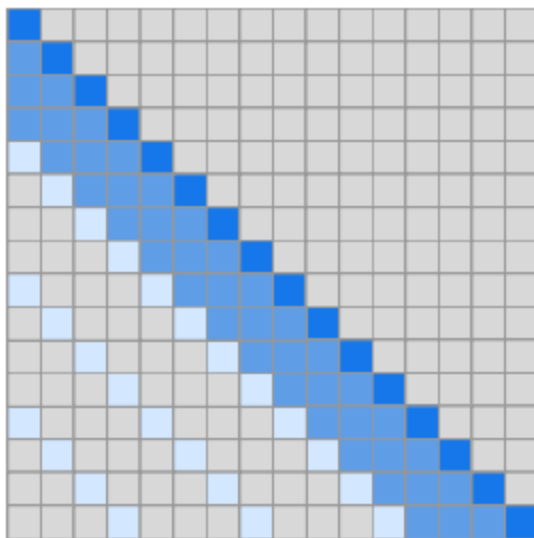


## Fixed Attention Patterns Example: Sparse Transformers, 2019.

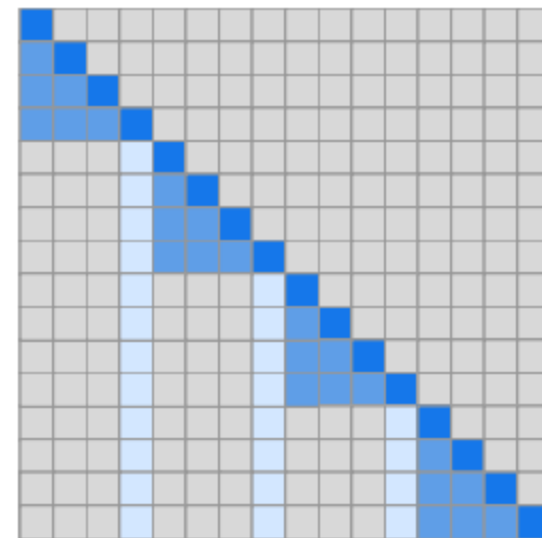
Consider attention during auto-regressive decoding. Manually mask attention computation (either adding strides or providing “anchors”)



(a) Transformer



(b) Sparse Transformer (strided)



(c) Sparse Transformer (fixed)





| Model / Paper                                                 | Complexity                  |
|---------------------------------------------------------------|-----------------------------|
| Memory Compressed <sup>†</sup> (Liu et al., 2018)             | $\mathcal{O}(n_c^2)$        |
| Image Transformer <sup>†</sup> (Parmar et al., 2018)          | $\mathcal{O}(n.m)$          |
| Set Transformer <sup>†</sup> (Lee et al., 2019)               | $\mathcal{O}(nk)$           |
| Transformer-XL <sup>†</sup> (Dai et al., 2019)                | $\mathcal{O}(n^2)$          |
| Sparse Transformer (Child et al., 2019)                       | $\mathcal{O}(n\sqrt{n})$    |
| Reformer <sup>†</sup> (Kitaev et al., 2020)                   | $\mathcal{O}(n \log n)$     |
| Routing Transformer (Roy et al., 2020)                        | $\mathcal{O}(n \log n)$     |
| Axial Transformer (Ho et al., 2019)                           | $\mathcal{O}(n\sqrt{n})$    |
| Compressive Transformer <sup>†</sup> (Rae et al., 2020)       | $\mathcal{O}(n^2)$          |
| Sinkhorn Transformer <sup>†</sup> (Tay et al., 2020b)         | $\mathcal{O}(b^2)$          |
| Longformer (Beltagy et al., 2020)                             | $\mathcal{O}(n(k + m))$     |
| ETC (Ainslie et al., 2020)                                    | $\mathcal{O}(n_g^2 + nn_g)$ |
| Synthesizer (Tay et al., 2020a)                               | $\mathcal{O}(n^2)$          |
| Performer (Choromanski et al., 2020)                          | $\mathcal{O}(n)$            |
| Linformer (Wang et al., 2020b)                                | $\mathcal{O}(n)$            |
| Linear Transformers <sup>†</sup> (Katharopoulos et al., 2020) | $\mathcal{O}(n)$            |
| Big Bird (Zaheer et al., 2020)                                | $\mathcal{O}(n)$            |

# Today's Learning Objectives

**Be able to answer:**

- What is an attention mechanism?
- What is a transformer architecture?
  - How does scaled-dot-product attention work?
  - What are positional encodings?
- What are the shortcomings of transformers?
  - What directions are being pursued for efficient transformer models?

